

Introduction to Client-Side Programming : JavaScript

1. Java Script란 ?
2. 기본 문법
3. Java Script 객체
4. Event Handler
5. 브라우져 내장 객체
6. 예제들
7. 참고 URL

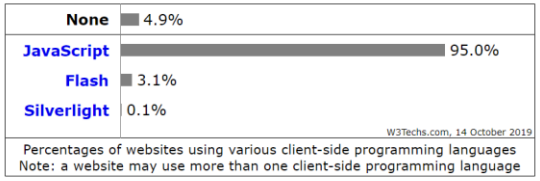
Usage of client-side programming languages for websites

This diagram shows the percentages of websites using various client-side programming languages. See technologies overview for explanations on the methodologies used in the surveys. Our reports are updated daily.

How to read the diagram:
4.9% of the websites use none of the client-side programming languages that we monitor.
JavaScript is used by 95.0% of all the websites.

Request an extensive market report of specific client-side programming languages.

Learn more



Percentages of websites using various client-side programming languages
Note: a website may use more than one client-side programming language

The following client-side programming languages have a market share of less than 0.1%

서강대학교
Sogang University

Market Share (24 Aug. 2026, https://w3techs.com/technologies)

Most popular server-side programming languages

© W3Techs.com	usage	change since 1 July 2026
1. PHP	70.2%	-0.6%
2. JavaScript	7.2%	+0.5%
3. Ruby	7.0%	+0.2%
4. Java	5.4%	-0.1%
5. Scala	5.1%	+0.1%

percentages of sites

Most popular client-side programming languages

© W3Techs.com	usage	change since 1 July 2026
1. JavaScript	98.9%	
2. Flash	2.8%	

percentages of sites

Most popular JavaScript libraries

© W3Techs.com	usage	change since 1 July 2026	market share	change since 1 July 2026
1. jQuery	66.5%	-1.5%	86.5%	-0.8%
2. Bootstrap	15.1%	-0.4%	19.7%	-0.2%
3. Underscore	7.1%	-0.2%	9.3%	-0.1%
4. React	6.1%		7.9%	+0.1%
5. Modernizr	5.2%	-0.2%	6.7%	-0.2%

percentages of sites

Most popular web servers

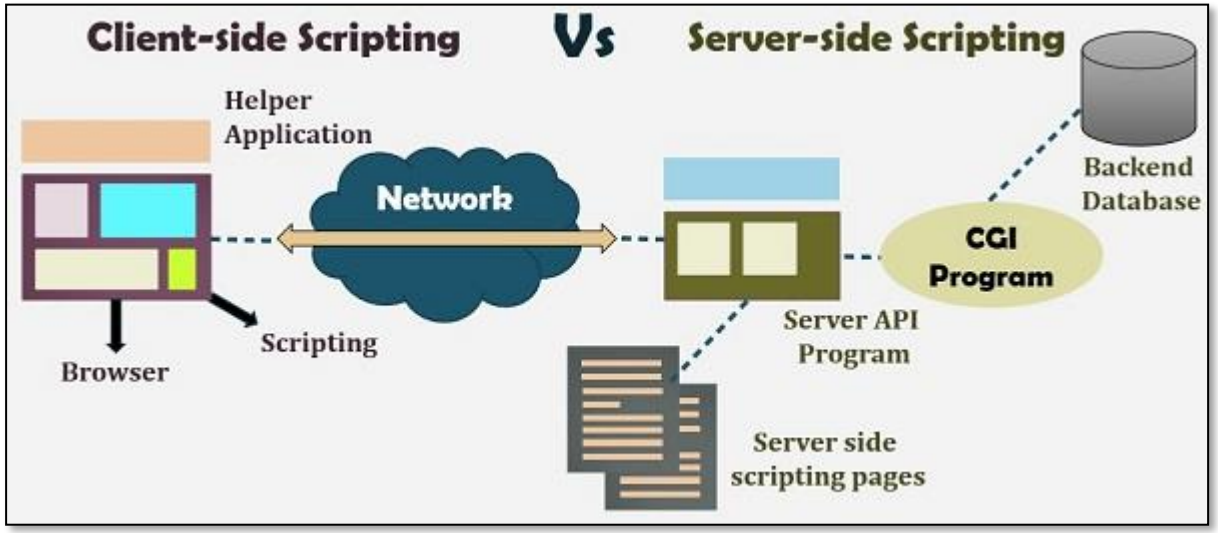
© W3Techs.com	usage	change since 1 July 2026
1. Nginx	31.4%	-0.4%
2. Cloudflare Server	29.6%	+1.1%
3. Apache	22.6%	-0.6%
4. LiteSpeed	14.9%	-0.2%
5. Node.js	7.0%	+0.4%

percentages of sites

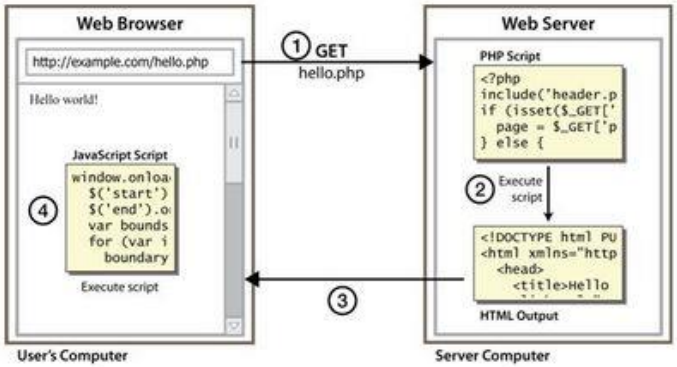
Most popular operating systems

© W3Techs.com	usage	change since 1 July 2026
1. Unix	92.0%	+0.3%
2. Windows	8.2%	-0.3%

percentages of sites

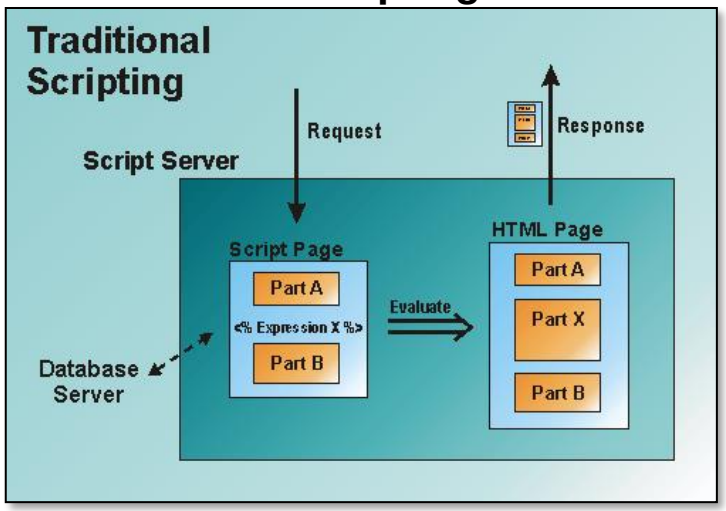


Client-side scripting



- **client-side script**: code runs in browser *after* page is sent back from server
often this code manipulates the page or responds to user actions

Server –Side Scripting



JavaScript: 웹 페이지에 동작을 더하는 언어

사용자 입력에 반응하고, 화면을 바꾸고, 서버와 데이터를 주고받는다

1 HTML · CSS · JavaScript의 역할

HTML
구조와 내용



웹 페이지의 뼈대와 내용을 만든다

CSS
모양과 배치



색, 폰트, 크기, 위치 등 화면을 꾸민다

JavaScript
동작과 상호작용



사용자 입력에 반응하고 화면을 동적으로 바꾼다

JavaScript는 ECMAScript 표준에 기반한 프로그래밍 언어이다.

2 웹에서 어떻게 실행되는가?



웹 서버
HTML, CSS, JS 파일 및 데이터 제공

HTML · CSS · JS 전달

JSON 응답 (데이터 응답)

웹 브라우저



HTML → DOM (문서 구조 생성)

JS 엔진에서 실행 (JavaScript 코드)

DOM / Web API 사용 (요소 조작, 이벤트 등)

화면 갱신 (변경 사항 반영)

클릭 · 입력 → 이벤트 처리
사용자의 클릭, 키보드 입력 동을 이벤트로 감지하고 실행

DOM · fetch · 타이머 등은 브라우저가 제공하는 API이다.

3 웹 환경의 주요 용도와 기능

이벤트 처리
클릭 · 키보드 · 스크롤에 반응하여 동작 실행

DOM 조작
텍스트 · 요소 · 스타일 변경 등으로 화면을 동적으로 업데이트

폼 입력 검사
필수 항목 · 입력 형식 확인 (서버에서도 검증 필요)

비동기 통신
JSON
fetch로 데이터를 받아 화면의 일부만 갱신

멀티미디어 · 시각화
영상 제어 · Canvas · 차트 · 게임 등 다양한 표현

브라우저 저장
localStorage 등에 설정 저장

검색 자동완성 | 장바구니 | 채팅 | 지도

4 작은 예제: 클릭하면 내용 변경

HTML + JavaScript 코드

```
1  <button id="btn">인사하기</button>
2  <p id="msg">대기 중</p>
3  <script>
4    document.querySelector("#btn")
5      .addEventListener("click", () => {
6        document.querySelector("#msg")
7          .textContent = "안녕하세요!";
8      });
9  </script>
```

실행 결과



이벤트 발생 (클릭) → JavaScript 실행 (이벤트 핸들러) → DOM 변경 (화면 업데이트)

5 언어의 특징과 실행 환경

변수 · 함수 · 객체로 로직 작성
값을 저장하고, 함수를 만들고, 객체로 데이터를 다룬다.

동적 타입 · Promise / async-await
변수의 타입이 실행 중에 결정되며, 비동기 작업을 간편하게 처리할 수 있다.



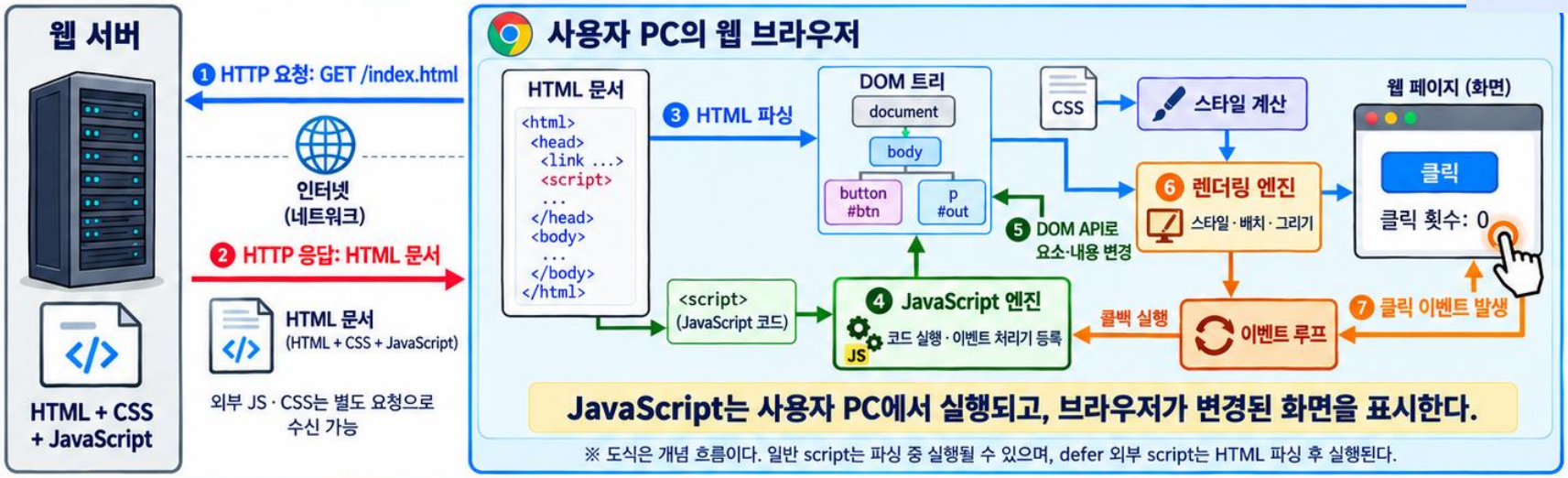
브라우저: V8 · SpiderMonkey · JavaScriptCore
주요 브라우저는 각자의 JavaScript 엔진으로 코드를 실행한다.

서버에서도 실행 가능: Node.js 등
브라우저 외에도 서버 환경에서 JavaScript를 실행할 수 있다.

HTML이 내용을 만들고, CSS가 꾸미고, JavaScript가 사용자와 상호작용한다.

웹 브라우저에서 JavaScript가 실행되는 원리

HTML로 화면을 구성하고, 이벤트에 반응하여 DOM과 화면을 갱신한다.



예제: HTML 안의 JavaScript

요소 아래에 script가 있으므로 실행 시점에 DOM 노드가 존재한다.

```html
<button id="btn">클릭</button>
<p id="out">클릭 횟수: 0</p>
<script>
  let count = 0;
  const btn = document.getElementById("btn");
  const out = document.getElementById("out");
  btn.addEventListener("click", () => {
    count += 1;
    out.textContent = "클릭 횟수: " + count;
  });
</script>
```

반복되는 상호작용



1. **클릭 → 이벤트 처리기 실행**
   사용자가 버튼을 클릭하면 미리 등록한 이벤트 처리기가 실행된다.
2. **count 증가 → 메모리의 상태 변경**
   변수 count 값이 0에서 1로 증가한다.
3. **textContent 변경 → DOM 갱신**
   p 요소의 textContent가 "클릭 횟수: 1"로 바뀐다.
4. **브라우저 렌더링 → 클릭 횟수: 1**
   변경된 내용을 바탕으로 화면이 갱신되어 사용자에게 표시된다.

이 예제는 추가 서버 요청 없이 동작

필요할 때만 서버와 통신

Fetch / AJAX로 데이터 요청

전체 페이지를 다시 불러오지 않고 일부 내용 갱신



HTML = 구조 CSS = 모양 JavaScript = 상태 · 이벤트 · 동작

Java vs JavaScript: 공통점과 차이점

이름은 비슷하지만, 서로 다른 언어와 실행 환경

공통점

- C 계열 문법(조건·반복)
- 객체 활용
- 자동 메모리 관리(GC)
- 현대 구현에서 JIT 활용

비교 관점	Java	JavaScript
수행 방식	.java → javac (컴파일러) → .class (바이트코드) → JVM (해석 실행 + JIT) 소스 → javac → 바이트코드 → JVM 일반적인 JVM: 해석 실행 + JIT	.js → JS 엔진 (파싱·실행) → (해석 실행 + JIT) 소스 → JS 엔진 → 파싱·실행 현대 엔진: 해석 실행 + JIT
언어 분류	• 정적 타입 · 클래스 기반 객체지향 • 범용 프로그래밍 언어	• 동적 타입 · 프로토타입 기반 • 객체지향 · 함수형 · 명령형 지원 class 문법도 제공
프로그램 저장·전달	.java, .class, JAR • .java: 소스 / .class: 바이트코드 • .class를 JAR로 묶어 배포 가능	.js, HTML • .js 파일 또는 HTML의 <script> • 브라우저에는 보통 소스 형태로 전달

타입 선언	`int n = 10;`
	`var x = 10; // 추론해도 타입 고정`
	Java: 컴파일 시 타입 검사 / JavaScript: 값의 타입이 실행 중 결정
		`let n = 10;`
		`n = 'ten'; // 다른 타입의 값 대입 가능`

입출력·Network	• JVM 환경: 콘솔 · 파일 · GUI • HTTP 클라이언트 · 소켓 제공	• 브라우저: DOM · 폼 · Canvas • fetch · WebSocket 등 Web API 브라우저 JS: CORS · 권한 · 파일 접근 제한 적용
브라우저와의 연계	• 현재 브라우저에서 Java. 직접 실행 미지원 • 과거: Applet + Java 플러그인 Java 플러그인은 JDK 11에서 제거	• 브라우저 내장 JS 엔진에서 실행 • DOM 조작 · 이벤트 처리로 HTML과 연계
활용 분야	• 웹 백엔드 · 기업 시스템 • Android 앱 · 데스크톱 프로그램	• 웹 프런트엔드 · SPA · 시각화 • Node.js 백엔드 · 모바일/데스크톱 앱

웹 애플리케이션에서는 함께 사용한다

JS 브라우저 / JavaScript
입력 처리 · DOM 갱신

HTTP 요청 →
← JSON / HTML 응답

서버 / Java 백엔드 (예: Spring)
업무 로직 · DB 접근

DB

Java는 JVM 중심, 브라우저 JavaScript는 JS 엔진 + Web API 중심

공식 자료 : Oracle Java · MDN JavaScript

```
<html>
 <head>
```

① client-side Javascript code

```
<script language=JavaScript>
        document.write("This is JavaScript<br>");
 </script>
</head>

<body bgcolor=white>
```

② client-sede HTML script

```
This is HTML<br>
```

③ client-side Javascript code

```
<script language=JavaScript>
        document.write("JavaScript is crazy language");
</script>
```

④ server-side php code

```
<?php
        echo “<H1> Hello World ! </H1>”
?>
 </body>
 </html>
```

cspro.sogang.ac.kr/~jhnang/cgi-bin/js_Hello.php

php Script on Server-side

```
<html>
 <head>
                                   ① client-side Javascript code
        <script language=JavaScript>
                document.write("This is JavaScript<br>");
        </script>
</head>

<body bgcolor=white>
                                   ② client-sede HTML script
        This is HTML<br>

                                   ③ client-side Javascript code
        <script language=JavaScript>
                document.write("JavaScript is crazy language");
        </script>

                                   ④ server-side php code
        <?php
                echo "<H1> Hello World ! </H1>"
        ?>
</body>
</html>
```

cspro.sogang.ac.kr/~jhnang/cgi-bin/js_Hello.php

PHP Interpreter

php Output (HTML file that is sent to Browser)

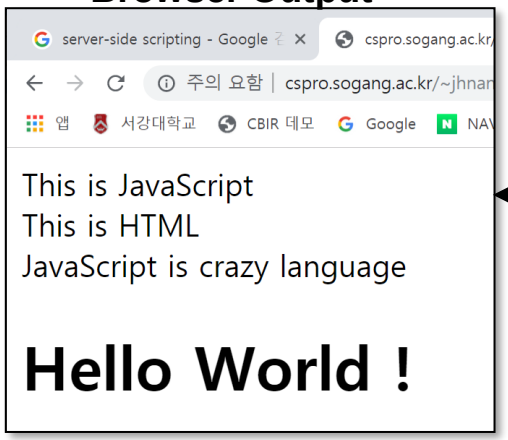
```
<html>
 <head>
                                   ① client-side Javascript code
        <script language=JavaScript>
                document.write("This is JavaScript<br>");
        </script>
</head>

<body bgcolor=white>
                                   ② client-sede HTML script
        This is HTML<br>

                                   ③ client-side Javascript code
        <script language=JavaScript>
                document.write("JavaScript is crazy language");
        </script>

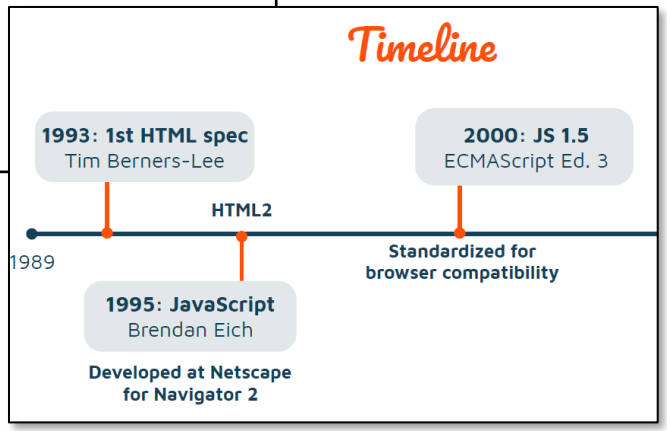
        <H1> Hello World ! </H1>
</body>
</html>
```

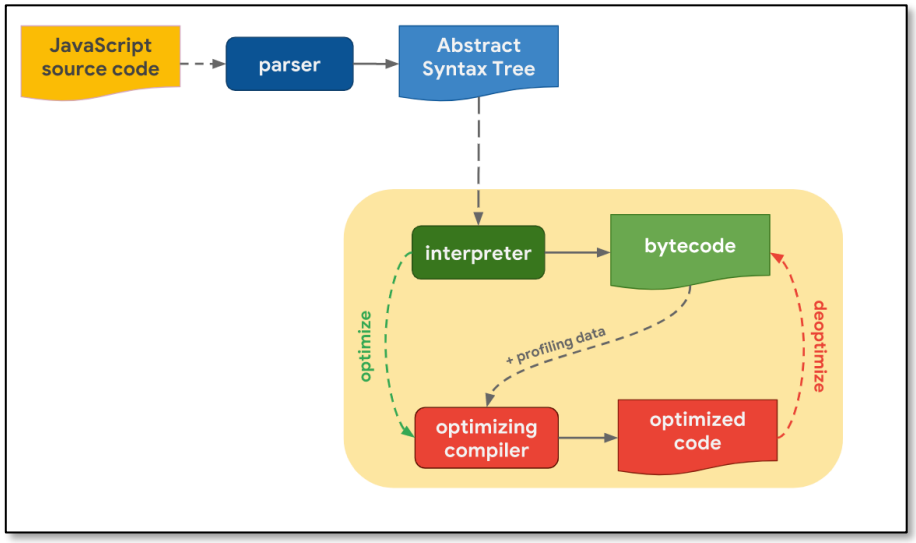
Browser Output

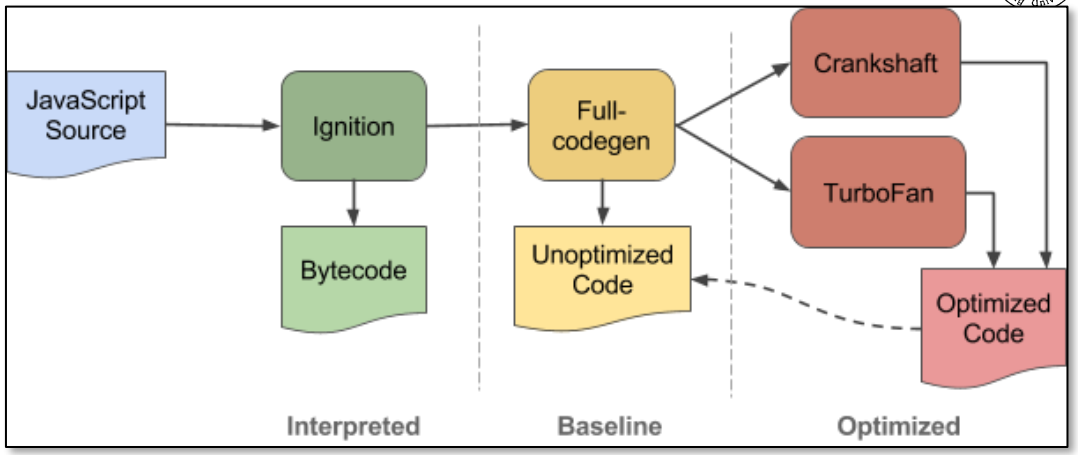


V8 (JavaScript engine)

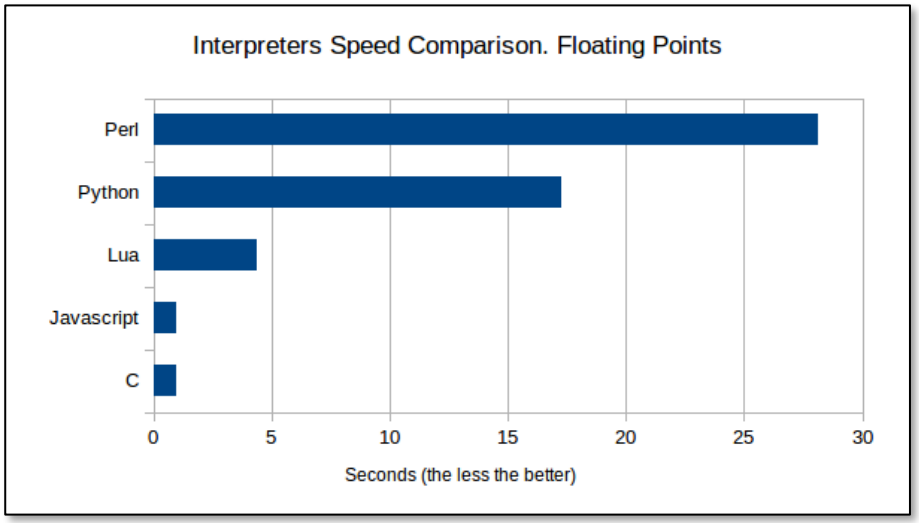
an open-source JavaScript engine (interpreter) developed by the Chromium Project for Google Chrome and Chromium web browsers.

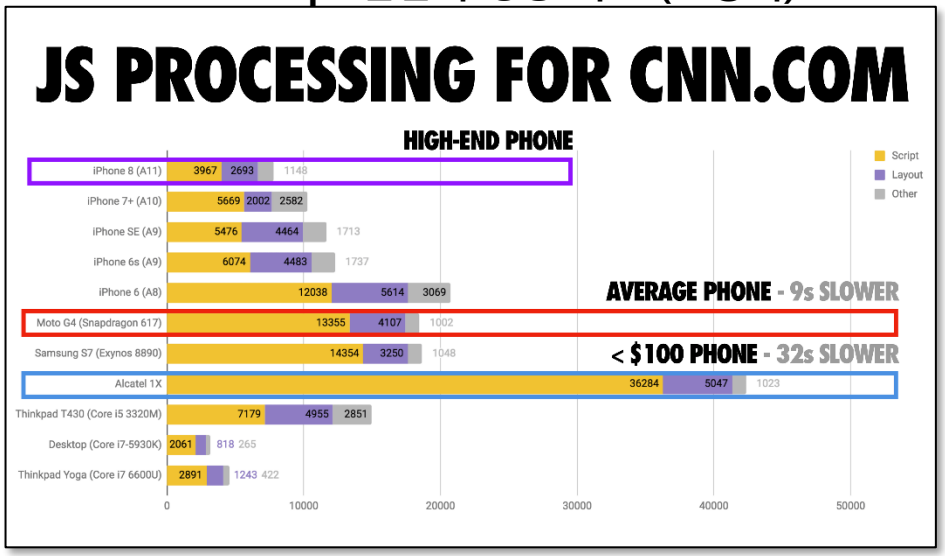






Javascript 엔진의 성능 비교 (노랑색)





웹 브라우저의 JavaScript 엔진 비교

같은 JavaScript 표준, 서로 다른 실행·최적화 구현



V8 | Google

Chrome · Edge · Chromium 계열

렌더링: Blink

- Ignition — 바이트코드 인터프리터
- Sparkplug — 기본 JIT
- Maglev — 중간 최적화 JIT
- TurboFan — 고급 최적화 JIT

가정이 깨지면 하위 단계로 복귀

특징: 단계별 컴파일 비용과 실행 속도 균형

SpiderMonkey | Mozilla

Firefox

렌더링: Gecko

- Baseline Interpreter — 바이트코드 실행
- Baseline JIT — 기본 기계어 생성
- Ion + Warp — 프로파일 기반 최적화

가정이 깨지면 하위 단계로 복귀

특징: CacheIR 피드백을 활용한 최적화

JavaScriptCore | WebKit

Safari

렌더링: WebKit

- LLInt — 바이트코드 인터프리터
- Baseline JIT — 기본 기계어 생성
- DFG JIT — 데이터 흐름 기반 최적화
- FTL JIT — 고급 최적화

가정이 깨지면 하위 단계로 복귀

특징: DFG 분석과 FTL 고급 코드 생성

대표적인 실행 계층: 실제 승격 경로는 버전·플랫폼·설정에 따라 달라짐.

공통 원리

JS 소스 → 파싱 → 바이트코드 → 실행 → 자주 실행되는 코드 JIT 최적화

실행 정보 수집 · 가비지 컬렉션(GC) · ECMAScript / WebAssembly 지원

비교할 지표

- 초기 실행 시간 (페이지 로드 후 바로 실행)
- 반복 실행 속도 (JIT 최적화된 코드 실행)
- 메모리 (힙 사용량, 메모리 효율)
- 전력 소비 (노트북 · 데스크톱 에너지 효율)

성능 순위는 코드 · 기기 · 버전에 따라 달라짐.

데스크톱 기준 · 모바일은 플랫폼에 따라 엔진이 다를 수 있음.

공식 문서: v8.dev · firefox-source-docs.mozilla.org · webkit.org

JIT 컴파일: 왜 필요하고, 어떻게 동작하는가?

Just-In-Time: 프로그램 실행 중 기계어를 생성하는 방식

1 동기: 빠른 시작과 빠른 반복 실행

인터프리터

바이트코드를 명령마다 해석 → 해석 · 분기 비용 반복 → 매번 해석하므로 반복 실행이 느림

JIT

컴파일 비용을 한 번 지불 → 생성한 기계어를 반복 실행 → 이후에는 기계어를 바로 실행하여 빠름

자주 실행되는 코드에서 컴파일 비용을 회수

컴파일 시간 · 메모리 증가 (초기 비용) ↔ 반복 실행 시간 감소 (반복 실행 시 이득)

2 방법: 실행 정보를 이용해 단계적으로 최적화

1. **JS 소스 → 파싱 → 바이트코드**
   - JS 소스
   - 파싱 (구문 분석)
   - 바이트코드 (중간 표현)
2. **인터프리터 / 기본 JIT로 실행**
   - 바이트코드를 해석하거나 간단히 컴파일하여 실행
3. **프로파일 수집: 호출 횟수 · 자료형 · 객체 구조**
   - 함수 호출 횟수 (Hot 여부)
   - 자료형 정보 (숫자, 문자열 등)
   - 객체 구조 (숨은 클래스 등)
   - 분기 패턴 등 실행 정보
4. **Hot code 선택**
   - 자주 실행되는 코드를 식별
5. **최적화 JIT → CPU 기계어**
   - 수집한 정보를 바탕으로 더 적극적으로 최적화하여 CPU용 기계어 코드를 생성
6. **기계어 재사용**
   - 0101 1010
   - 생성한 기계여를 반복 실행하여 높은 실행 성능을 달성

기본 JIT: 빠른 코드 생성
최적화 JIT: 더 많은 분석

3 예제: 자료형을 관찰하고, 가정이 맞는 동안 빠르게 실행

JavaScript 코드

```
1 function add(a, b) {
2   return a + b;
3 }
4
5 add(1, 2);  // 3
6 add(3, 4);  // 7
```

실행 관찰

숫자 입력이 반복됨

add(1, 2)
add(3, 4)
...

최적화된 기계어 (개념도)

Guard: 숫자 타입 확인 → 숫자 덧셈 기계어 실행

가정이 유지되는 경우 (숫자 입력) → 가정 유지 → 빠른 실행
Guard를 통과하면 최적화된 기계어를 계속 실행

가정이 깨지는 경우 (다른 자료형)

add('A', 'B'); // 'AB' → Guard 실패 → Deoptimization 안전한 실행 단계로 복귀

문자열 + 는 연결 연산: JS의 의미를 보존해야 함

주요 최적화 기법 (예시)

- **함수 인라이닝** — 작은 함수를 호출 없이 직접 확장
- **자료형 특화** — 관찰한 자료형에 맞는 특화된 코드 생성
- **불필요한 연산 제거** — 도달할 수 없는 코드, 중복 계산 등 제거

대표 실행 계층 (원본 그림과의 연결)

V8 | Google	SpiderMonkey | Mozilla	JavaScriptCore | WebKit
Ignition → Sparkplug → Maglev → TurboFan	Baseline Interpreter → Baseline JIT → Ion / Warp	LLInt → Baseline JIT → DFG → FTL

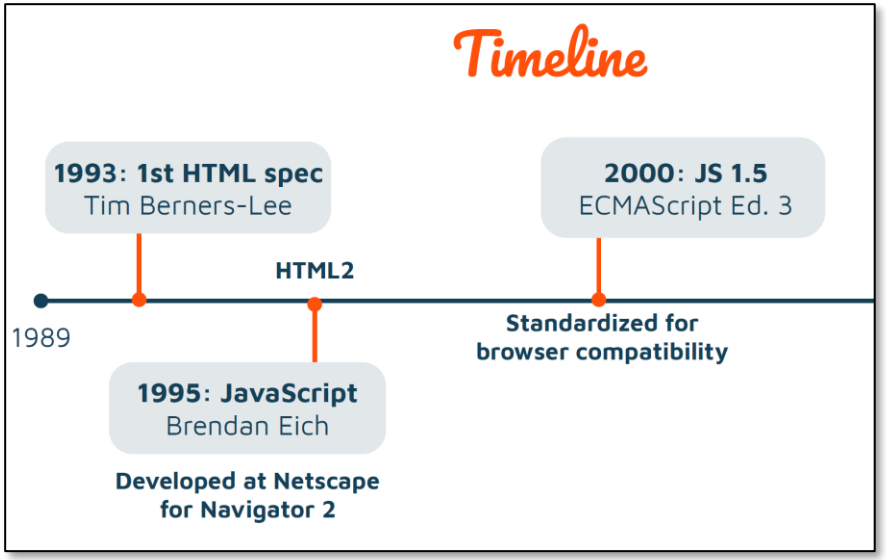
승격 · 복귀 경로는 엔진 · 버전 · 설정에 따라 달라짐. 모든 코드가 최상위 JIT까지 컴파일되는 것은 아님.

1. JavaScript란 ?

- JavaScript란 ?
  - Netscape사에서 개발한 일종의 Web 페이지용 프로그래밍 언어
  - Java와 HTML을 연결할 목적으로 만들어짐
    ⇔ Live Script -> Java Script (SUN의 승인 받음)
  - Sunsoft사의 Java 언어를 Web 페이지의 필요성에 맞추어서 단순화시킨 script 언어
    ⇔ (cf) Script 언어란 ?
    ⇒ 일반적인 프로그래밍 언어와는 달리 기존의 기능 (명령어, 객체)을 조합하여 새로운 기능을 제공할 수 있도록 하는 언어
    ⇒ 일반적으로 인터프리터에 의하여 수행됨
    ⇒ 일반적인 프로그래밍 언어보다는 프로그래밍하기 쉬움
    ⇒ 예) JavaScript, VisualBasic, Perl,....
  - HTML 문장 안에서 <Script=JavaScript>와 </Script>태그 사이에 삽입되어 사용자가 페이지를 이동하거나 이미지를 클릭하고 Form문에 입력한 내용을 체크할 수 있도록 사용자와 상호 작용하는 인터프리터 언어이다.
  - 클래스를 지원하지는 않지만 Built-in Object를 지원 한다.
  - HTML문서 자체에 포함되어 실행된다.
    ⇔ 컴파일 과정을 거치지 않고 브라우저에서 실행 된다.
  - 플랫폼 독립적인 스크립트 언어
    ⇔ Google Chrome, MS사의 Explore에서 지원

Interactiveness
without reloading

- **JavaScript**
  - popular scripting language on the Web, supported by browsers
  - separate scripting from structure (HTML) and presentation (CSS)
  - client- and server-side programming
  - object-oriented, imperative, functional



- **How to embed JS in HTML**
  - Inline in HTML

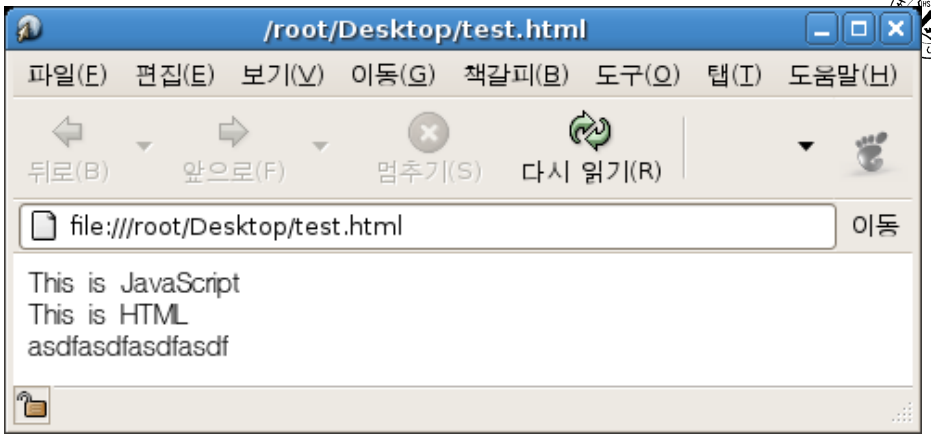
```
<script type="text/javascript">
  Javascript goes here...
</script>
```

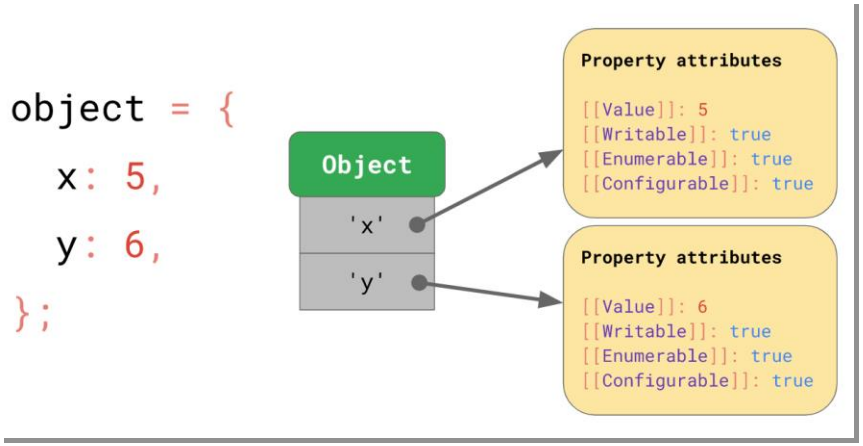
  - Embed external file

```
<script type="text/javascript" src="code.js"></script>
```

• JavaScript 구성 예 (ex1)

```
<html>
 <head>
 <script language=JavaScript>
 <!-- Begin of JavaScript
 document.write("This is JavaScript<br>");
 // End of Script -->
 </script>
 </head>
 <body bgcolor=white>
 This is HTML<br>
 <script language=JavaScript>
 <!-- Begin of JavaScript
 document.write("JavaScript is crazy language");
 // End of Script -->
 </script>
  </body>
  </html>
```





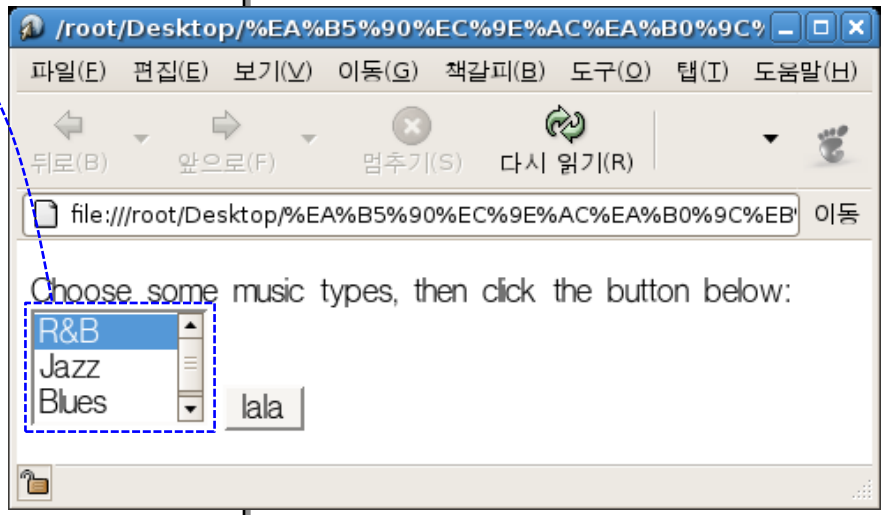
- **함수 (function)와 메소드 (method)**
  - ⇒ 함수 : 스크립트 작성자가 특정한 기능을 수행하도록 쓰여진 기능적인 블록
  - ⇒ 메소드 : 객체가 가지고 있는 특정한 작업을 수행하는 고유한 함수를 지칭

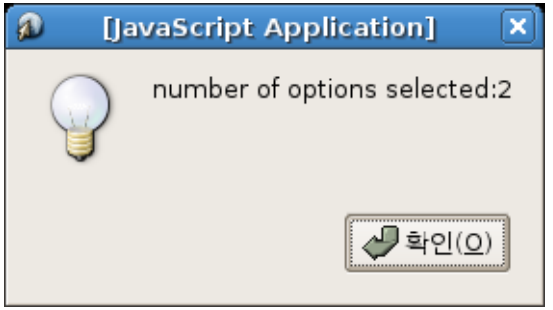
- **객체란 ?**
  - ⇒ 고유한 이름 (name, variable), 속성 (attribute, property), 그리고 행동 (behavior, method)을 가진 것
  - ⇒ built-in 객체 vs. 사용자 정의 객체

Ex2.html

```
<HTML>
<HEAD>
<SCRIPT LANGUAGE="JavaScript">
    function howMany(selectObject) {
        var numberSelected=0
        for(i=0;i<selectObject.options.length;i++) {
            if(selectObject.options[i].selected==true)
                numberSelected++
        }
        return numberSelected
    }
</SCRIPT>
</HEAD>
<BODY>
<FORM NAME="selectForm">
<P>Choose some music types, then click the button below:</B>
<BR><SELECT NAME="musicTypes" SIZE="3" MULTIPLE>
<OPTION SELECTED> R&B
<OPTION> Jazz
<OPTION> Blues
<OPTION> New Age
<OPTION> Classical
<OPTION> Opera
</SELECT>
<INPUT TYPE=button VALUE="lala" NAME="How many are selected?"
onClick="alert ('number of options selected:'
                + howMany(document.selectForm.musicTypes))">

</FORM> </BODY> </HTML>
```





2. 기본 문법

- 값 (Value)
  - 변수(Variables)는 다음과 같은 값을 가질 수 있다.
    - ⇔ 12, 3.14192와 같은 숫자
    - ⇔ true나 false와 같은 부울(boolean)형
    - ⇔ "안녕?"과 같은 문자열
    - ⇔ null로 쓰이는 널(null)형
  - 형변환

```
var I = 42
var I = "문자열의 값"
x = "The answer is " + 42
```

    - ⇔ Built-in 함수들
      - ⇒ eval() : 숫자형으로 변환해 주고 값을 구해 주는 함수
      - ⇒ parseInt() : 문자열을 정수값으로 변환시켜 주는 함수
      - ⇒ parseFloat() : 문자열을 부동소수점형으로 변환해 주는 함수

VARIABLES

Dynamically typed: types associated with values, not with variables

Use **var** to define local variables

variables defined implicitly through assignment have global scope

- 변수명 (Variable Name)
  - 식별자나 이름은 반드시 문자와 밑줄("_")로 시작해야 한다.
  - 대소문자 구별(Case Sensitive)을 한다.
  - 변수명의 예 (Abc12, Number_hits, temp99 , _name)
  - 변수는 어디에서나 선언될 수 있다. (전역 변수, 지역 변수)

- **Literals**
  - ⇔ Literals는 자바스크립트에서의 값을 표현하는 방법이다.
    - ⇒ 42, 3.14159, "To be or not to be"
- **정수형 (Integers)**
  - ⇔ 정수형은 10진수(decimal), 16진수(hexadecimal)나 8진수(octal)의 형식으로 표현할 수 있다.
    - ⇒ 10진수: 0-9 사이의 값을 갖는다.
    - ⇒ 16진수: 0-9, A-F(혹은 a-f) 사이의 값을 갖는다. 앞에 0X(혹은 0x)가 붙는다.
    - ⇒ 8진수: 0-7 사이의 값을 갖는다. 앞에 0이 붙는다.
- **부동소수점형 (Floating Point)**
  - ⇔ 보통 실수형으로 소수점의 값을 갖는다. 지수형(exponent)의 형식을 가질 수도 있다. 적어도 하나이상의 숫자와 소수점이나 지수형을 가져야 한다.
    - ⇒ 3.1415, -3.1E12, .1e12, 2E-12
- **부울형(Boolean)**
  - ⇔ 참(true)과 거짓(false)의 두 가지 값을 가진다.
- **문자열형(String)**
  - ⇔ null값이나 겹따옴표(")나 따옴표(')로 둘러 쌓인 문자들을 가진다.
    - ⇒ "blah", 'blah' , "1234", "one line ₩n another line"
- **특수 문자(Special Characters)**
  - ⇔ ₩b, ₩f, ₩n, ₩r, ₩t

- **연산자**

연산자 구분	연산자
지정 연산자	=, -=, +=, *=, /=
비트 연산자	<<, >>
수치 연산자	+, -, *, /, %, ++, --
논리 연산자	&, |, ^
비교 연산자	==, !=, <=, >=, <, >
스트링 연산자	+
객체 생성 연산자	new

- **제어문 (Control Statements)**

문장	문 법
조건문	if (조건) { 문장1} else {문장2 }
반복문	for (초기식;조건식;증감식) { 문장 } while (조건식) { 문장 }
객체 관리 문장	for (변수 in 객체) { 문장 } while (객체) { 문장 }
설명문	// 설명문 /* 설명문 */

조건문

```
function checkData (){
   if (document.form1.threeChar.value.length == 3) {
      return true}
   else {
         alert("Enter exactly three characters. " +
               document.form1.threeChar.value + " is not valid.")
         return false}
   }
```

for문

```
function howMany(selectObject) {
     var numberSelected=0
     for(i=0;i<selectObject.options.length;i++) {
          if (selectObject.options[i].selected==true)
          numberSelected++
      }
     return numberSelected
}
```

while/break문

```
function testBreak(x) {
     var i = 0
     while (i < 6) {
        if (i == 3) break
        i++
     }
     return i*x
  }
```

with

```
var a, x, y
var r=10
with (Math) {
   a = PI * r * r
   x = r * cos(PI)
   y = r * sin(PI/2)
}
```

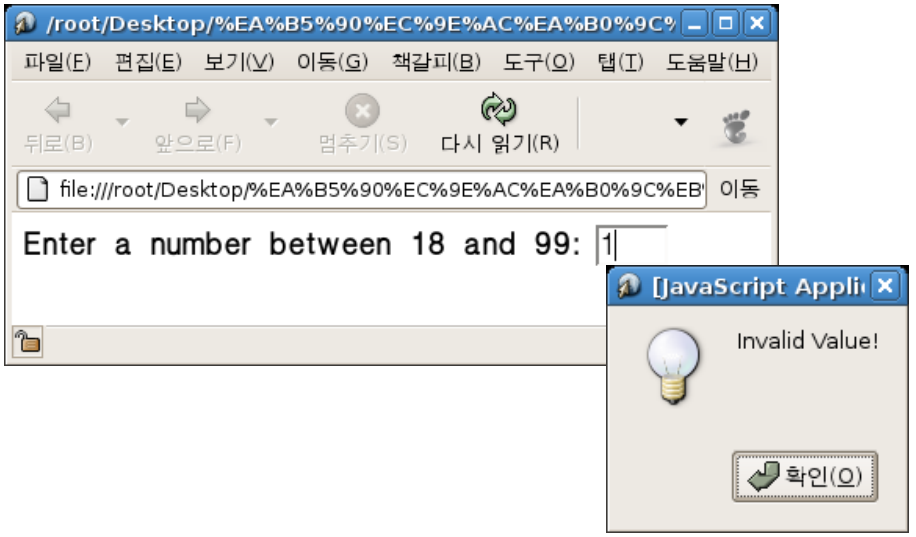
this

Ex3.html

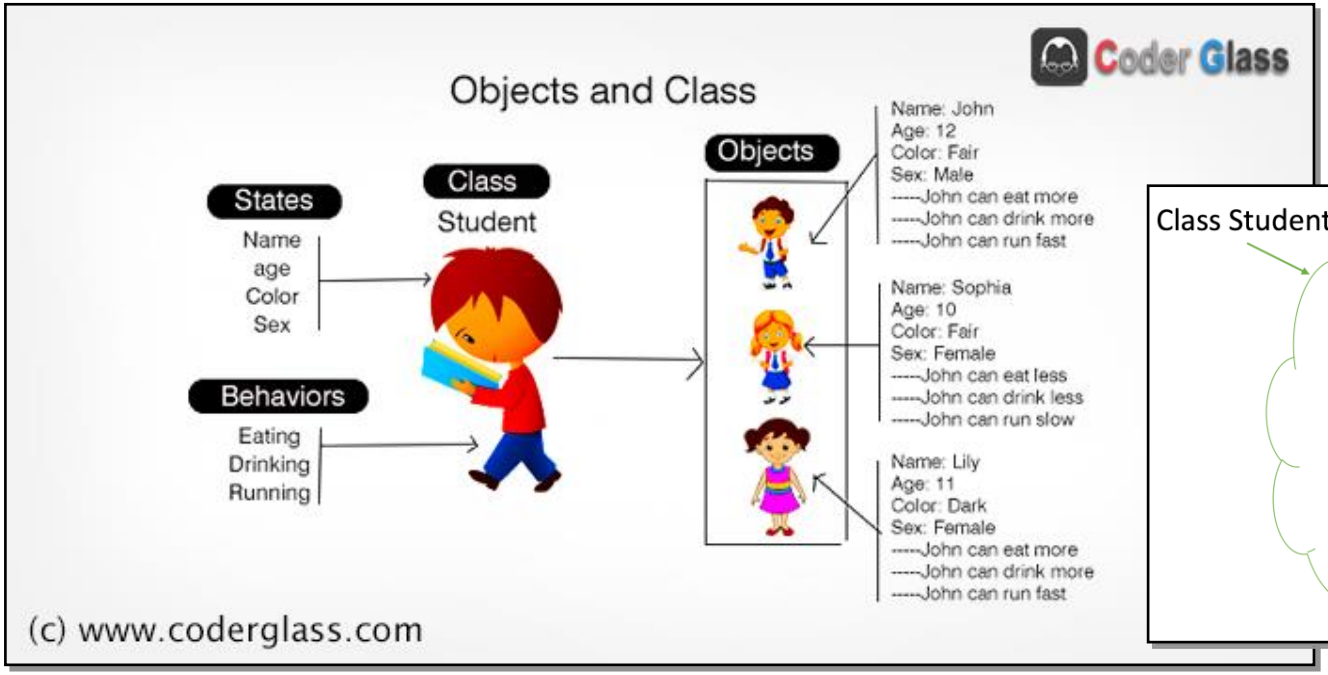
```
<html>
 <SCRIPT language=JavaScript>
 function validate(obj, lowval, hival) {
    if ((obj.value < lowval) || (obj.value > hival))
       alert("Invalid Value!")
 }
 </SCRIPT>
 <body>
 <B>Enter a number between 18 and 99:</B>
 <INPUT TYPE = "text" NAME = "age" SIZE = 3
       onChange="validate(this, 18, 99)">
 </body>
 </html>
```

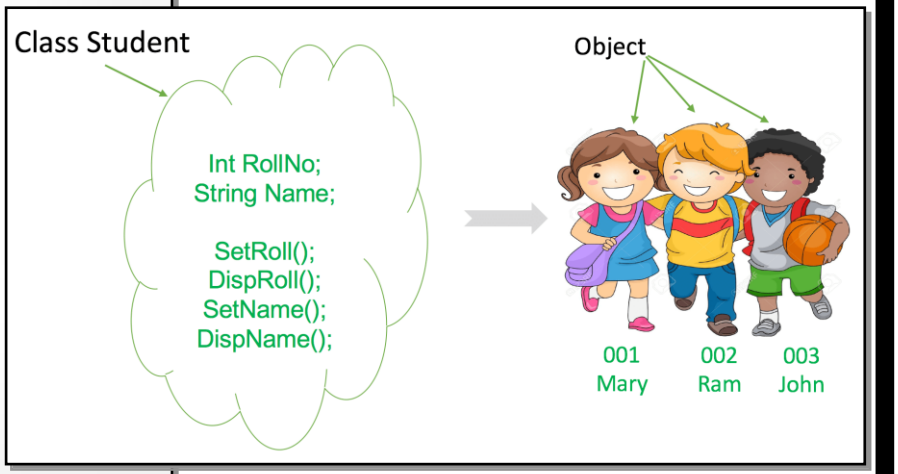
Event Handler

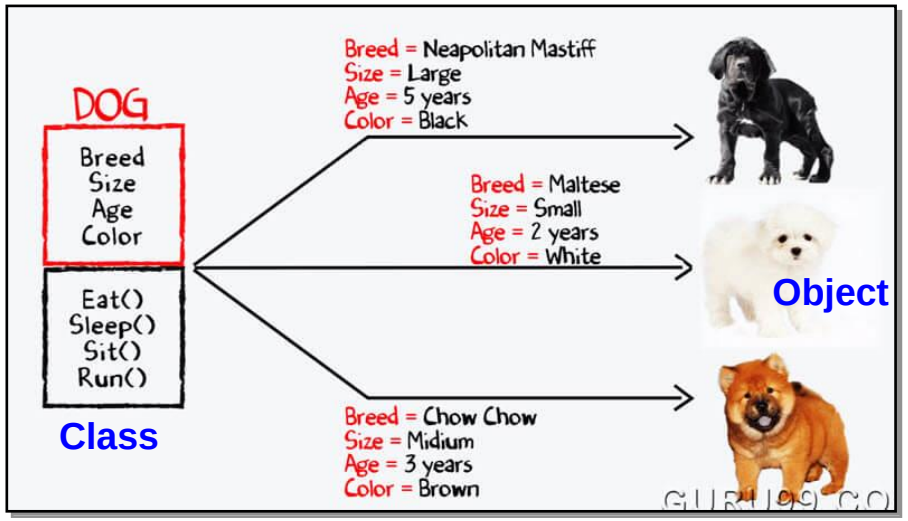
```
validate(document.input[0].age, 18, 99)
```

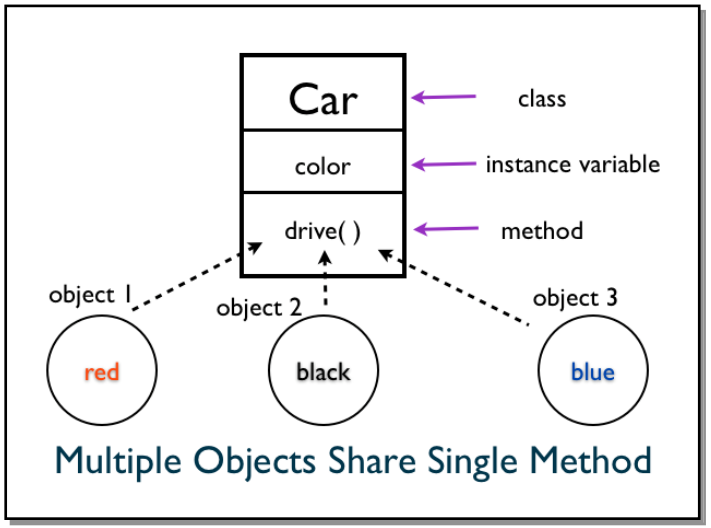


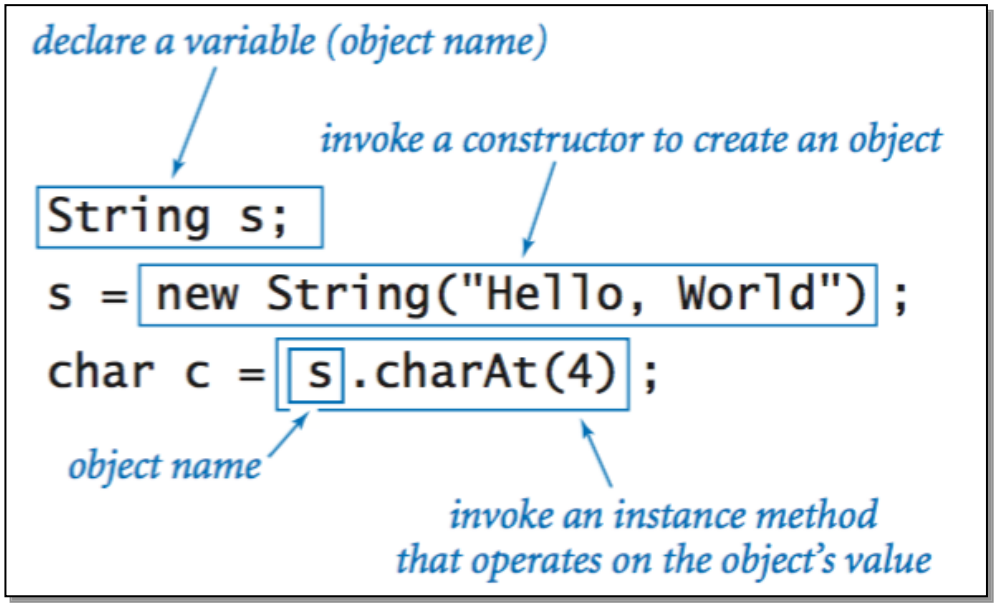
3. JavaScript 객체

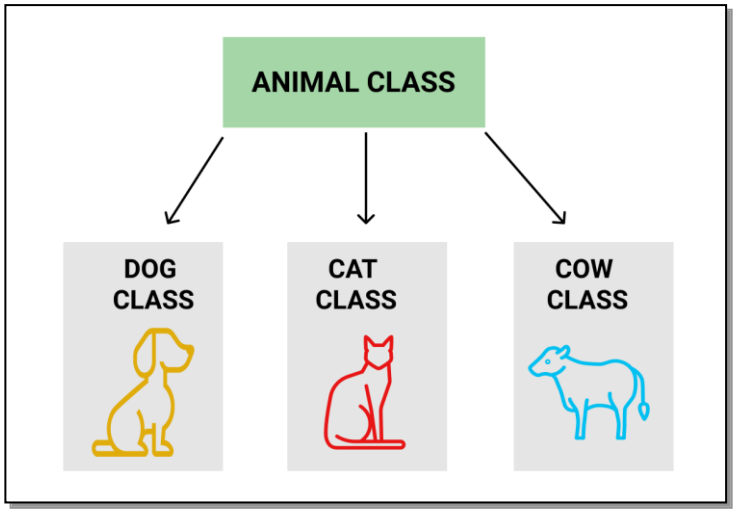




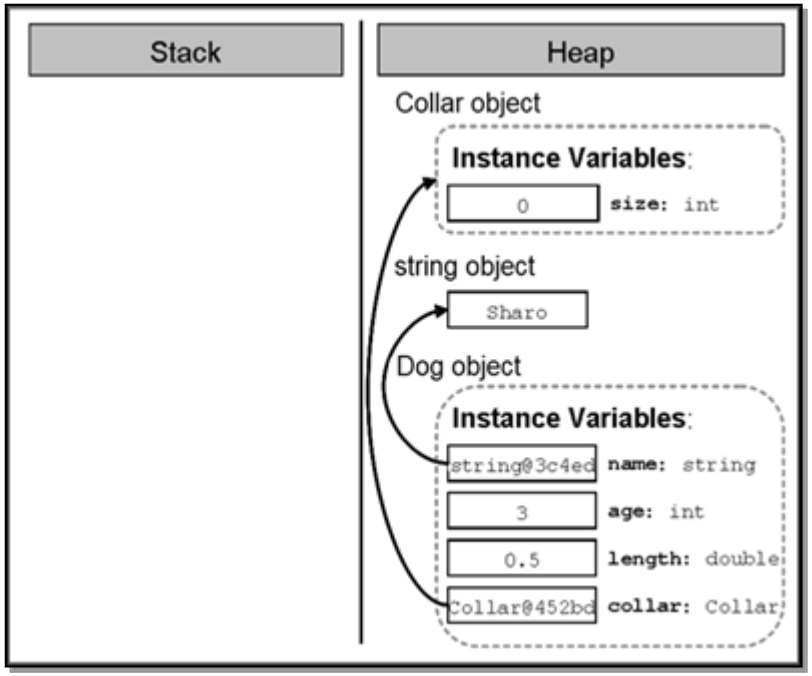


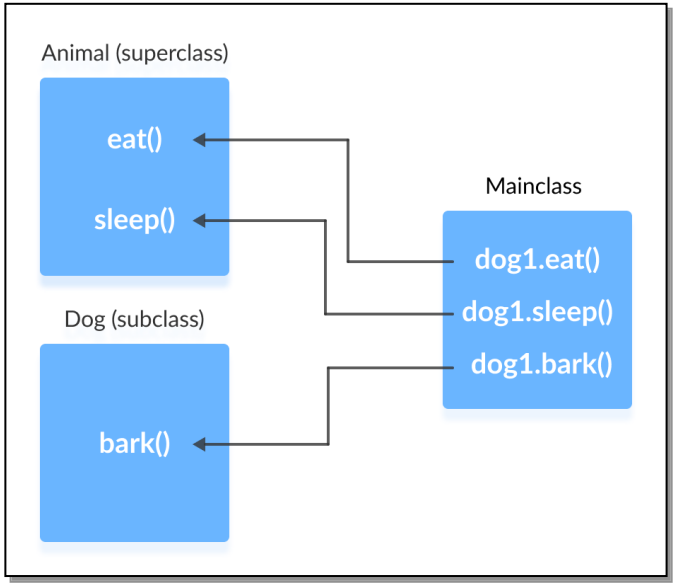


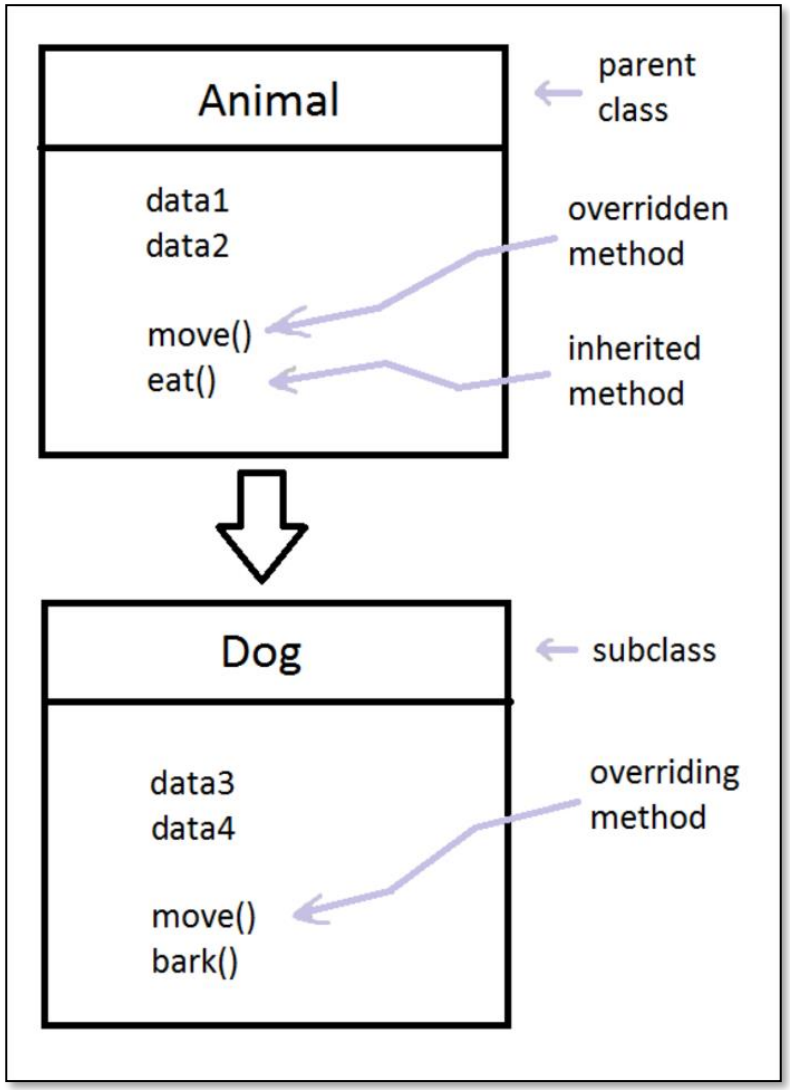




Inheritance (상속)







Object Oriented Programming in C++

Example of Inheritance

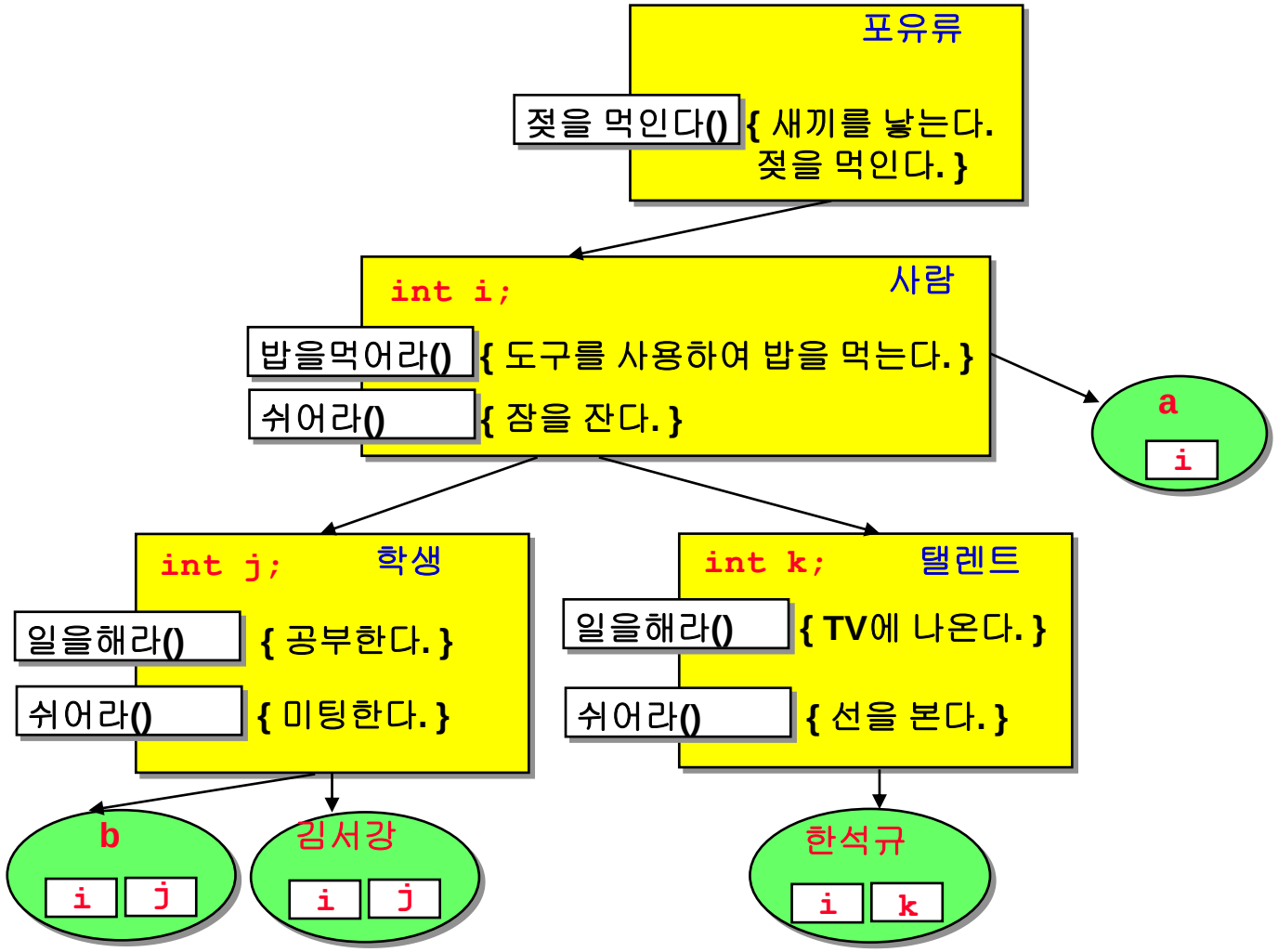
```
class Animal {
        public:
                int legs = 4;
};
class Dog : public Animal {
        public:
                int tail = 1;
};
int main() {
        Dog d;
        cout <<"Legs are: "<<d.legs<<endl;
        cout << "Tail is: "<<d.tail;
}
```

```
Legs are: 4
Tail is: 1
```

Lecture Slides By Adil Aslam

Object Creation

- **C++ :**
  - ✓ `Dog d;` /* 선언에 의하여 객체 생성 */
- **Java, JavaScript :**
  - ✓ `Dog d; d = new Dog();`
  - ✓ 선언은 객체 포인터 생성, new에 의하여 객체 생성




```
int i;
사람 a;
학생 김서강, b;
탤런트 한석규;

b = a; /* ? */

a.쉬어라();
a = 김서강;
a.쉬어라();
a = 한석규;
a.쉬어라();
```

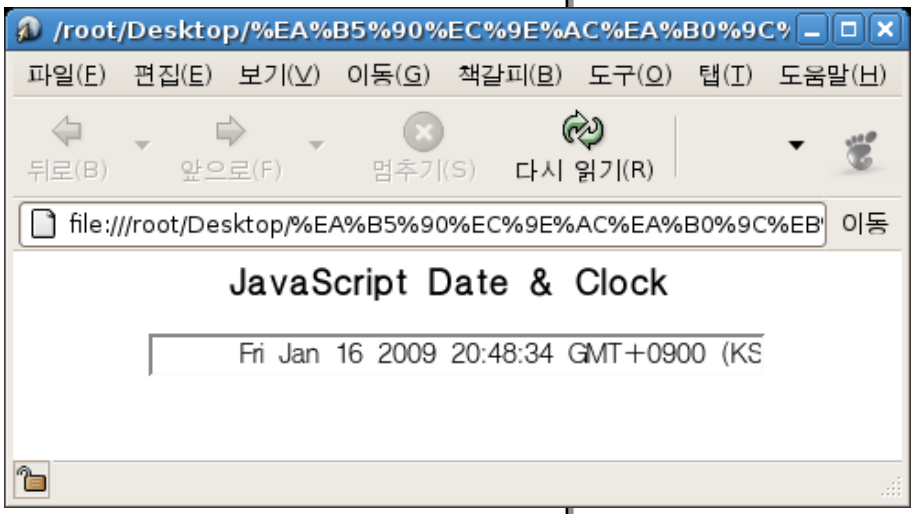
- 객체 (Object)
  - 자바 스크립트에서는 20여개의 Object를 기본으로 지원한다. Object는 Property 와 Method를 가진다.
    - ⇔ Properties : Object에 속해 특정 값을 저장 하는 변수/데이터
    - ⇔ Method : Java Script의 Object에 속해 있어 특별한 기능을 수행하는 함수
  - JavaScript Object는 계층(hierarchy)을 갖고 있으며 Object들은 Object의 이름에 의해 참조되고, 상위계층의 오브젝트 이름 뒤에 "."을 붙여서 하위 계층임을 표시 할 수 있다
    - ⇔ objectname.methodname
  - 속성(Properties)
    - ⇔ JavaScript의 Object는 Properties(속성)를 가지고 있다
    - ⇔ Property는 그 값을 지정함으로써 정의할 수 있다.
    - ⇔ '.'으로 상위 Object와 구분한다
      - ⇒ car.make = "Ford" car.model = "Mustang" car.year = 67
    - ⇔ 배열과 Java Script의 Properties는 실제적으로는 연관성이 있다. 사실상 Properties와 배열은 서로 같은 값을 표현하는 방법의 차이이다
      - ⇒ car[0] = "Ford" car[1] = "Mustang" car[2] = 67

- **Built-in 객체 및 함수**
  - **Date 객체**
    - ⇔ 날짜와 시간에 관련된 객체
    - ⇔ 형식 : Today = new Date()
    - ⇔ methods
      - ⇒ getYear(), setYear(), getMonth(), setMonth(), getDate(), setDate()
      - ⇒ getDay(), setDay(), getHours(), setHours(), getMinutes(), setMinutes()
      - ⇒ getSeconds(), setSeconds()
      - ⇒ parse(날짜문자열)
      - ⇒ toGMTString(), toLocalString(), toString()
  - **String 객체**
    - ⇔ property
      - ⇒ length : 문자의 객수
    - ⇔ methods
      - ⇒ big(), small(), blink(), bold(), fixed(), italic(), sub(), sup(), fontcolor(색),
      - ⇒ fontsize(크기)
      - ⇒ anchor("표식지정") // <A NAME="표식" > 글자 </A>
      - ⇒ link("위치") // <A HREF="위치"> 글자 </A>
      - ⇒ charAt(숫자), indexOf("문자열"), lastIndexOf("문자열"),
      - ⇒ subString(숫자, 숫자), toLowerCase(), toUpperCase()

Clock1.html

```
<HTML>
<HEAD>
<SCRIPT language="JavaScript">
var enabled = 0;
function TOfunc() {
  TO = window.setTimeout( "TOfunc()", 1000 );
  var today = new Date();
  var todaystring = today.toString();
  var i;
  var fin = "";
  var offset = (50  - todaystring.length) / 2;

  if(offset < 0) offset = 0;
  for(i=0;i<offset;i++) fin += " ";
  fin += todaystring;
  document.forms[0].elements[0].value = fin;
  //  document.forms[0].disp.value = fin;
}
</SCRIPT>
</HEAD>
<BODY BGCOLOR=WHITE onLoad="TOfunc()">
<CENTER><font size=4 face="arial"><b>JavaScript Date & Clock</b></font>
<BR><BR>
<FORM>
<input type="text" name="disp" value="" size=40>
</FORM>
</CENTER>
</BODY>
</HTML>
```

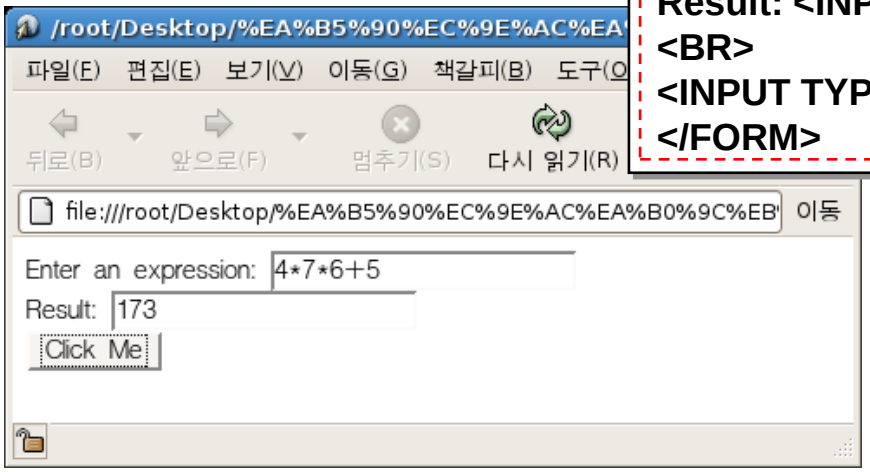


<실습 1> 2초씩 수정되는 Clock 실습

- Math 객체
  - ⇔ 상수
    - ⇒ Math.PI // (예) document.write("E : " + Math.E + "
")
  - ⇔ methods
    - ⇒ sin(x), cos(x), tan(x), asin(x), acos(x), atan(x), atan2(x,y), abs(x), exp(x)
    - ⇒ log(x), pow(x,y), sqrt(x), random(), round(x), floor(x), ceil(x),
    - ⇒ max(x,y), min(x,y)
- Built-in 함수
  - ⇔ eval()
  - ⇔ parseInt()
  - ⇔ parseFloat()

Evalform

```
<SCRIPT>
function compute(obj) {
    obj.result.value = eval(obj.expr.value)
}
</SCRIPT>
<FORM NAME="evalform">
Enter an expression: <INPUT TYPE=text NAME="expr" SIZE=20 >
<BR>
Result: <INPUT TYPE=text NAME="result" SIZE=20 >
<BR>
<INPUT TYPE="button" VALUE="Click Me" onClick="compute(this.form)">
</FORM>
```



this :
1) 객체 함수인 method안에서 this는 method를 포함하는 객체
2) 일반 함수인 function안에서 this는 전역객체 (document 객체)를 의미 함.

- In Javascript 'this' always refers to the "owner" of the function we're executing ➔ 각 함수의 owner는 누구인지 확인…

4. Event Handler

- Event
  - 특정한 동작이 발생하였다는 신호
  - Event의 종류

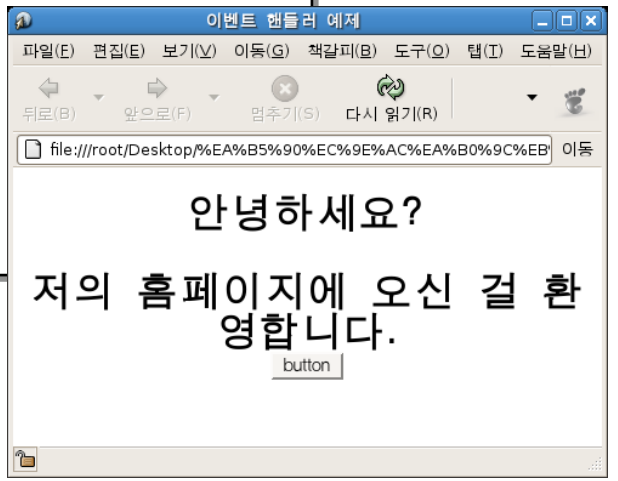
Event	발생 원인
blur	입력양식 필드에서 포커스가 다른 곳으로 이동했을 때
click	입력양식이나 링크를 마우스로 클릭했을 때
change	입력양식필드에 있는 값을 바꾸었을 때
focus	입력양식필드로 포커스가 들어왔을 때
load	브라우저에 문서가 읽혀졌을 때
mouseover	링크위로 마우스가 지나갔을 때
select	입력양식의 한 필드가 선택되었을 때
submit	입력양식을 서버로 보낼 때
unload	브라우저에서 문서가 없어졌을 때
abort	이미지를 읽다 중단시켰을 때
error	문서나 이미지를 읽다가 에러가 발생했을 때
mouseout	마우스가 링크나 특정 영역 안에 있다가 나갔을 때
reset	입력양식에서 리셋시켰을 때

- **Event Handler**
  - 이벤트가 발생하였을 때 마다 호출되는 함수 혹은 메소드
  - 지정 방법
    - ⇔ 이벤트 헨들러는 이벤트 핸들러를 사용하고자 하는 곳에서 HTML Tag의 한 속성으로 지정할 수 있다.
    - ⇔ 속성의 이름은 해당 이벤트 이름 앞에 on을 붙이면 된다.
      - ⇒ <input type="button" value="버튼" onclick="stopTime()">
      - ⇒ 리눅스 계열의 브라우저에서는 보안상의 이유로 onLoad등이 동작하지 않음

EventHandler

```
<html>  <head>
 <title> 이벤트 핸들러 예제 </title>
 </head>
 <body>  <center>
 <h1>안녕하세요?
 <p> 저의 홈페이지에 오신 걸 환영합니다.<br>
<input type="button" value="button"
                                onclick="alert('반갑습니다')">
 </h1>
 </center>
 </body>  </html>
```

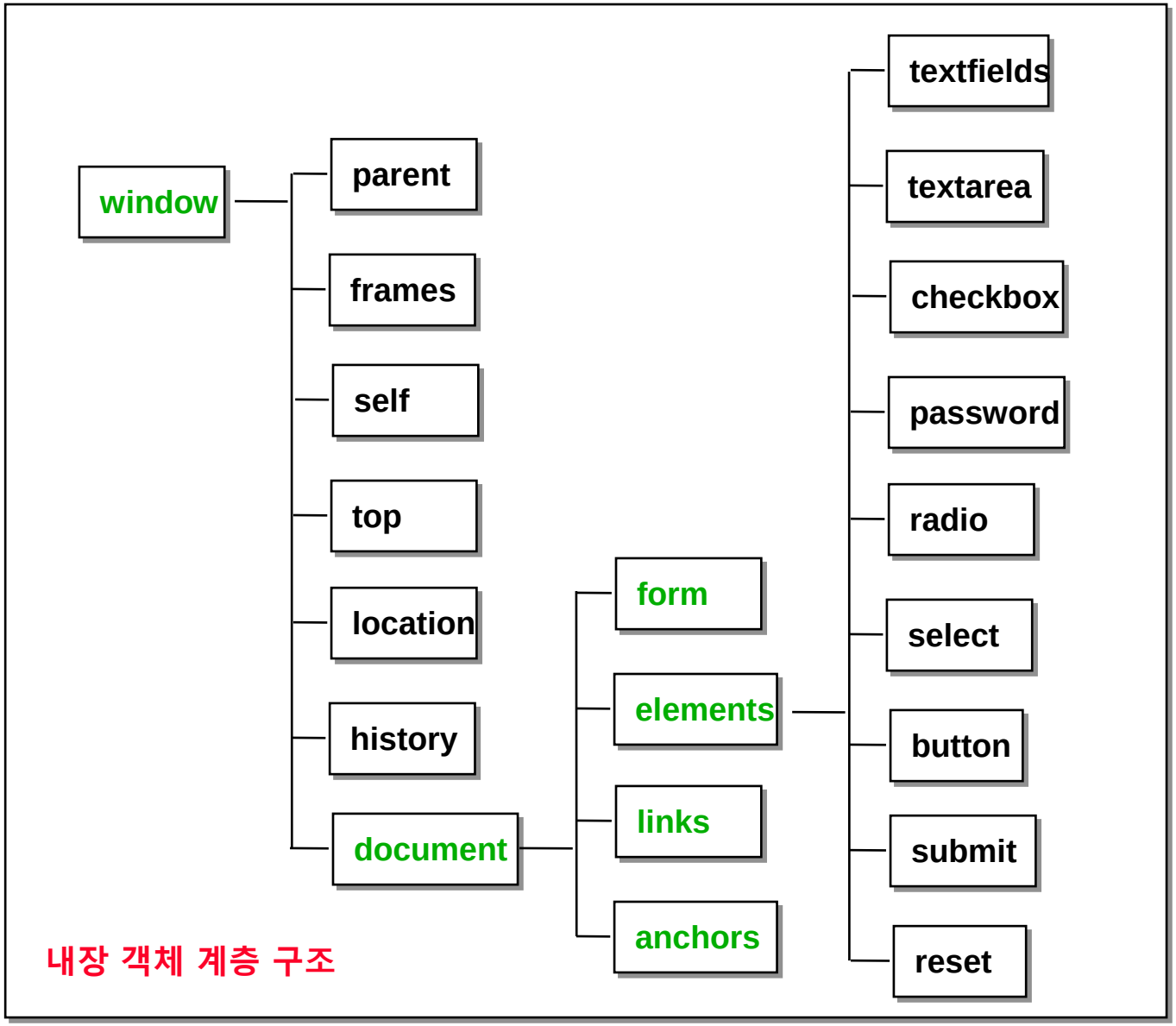




연습 문제 : <누적 합 기능을 추가한 계산기> (예제 10)

5. 브라우저 내장 객체

window.document.form[“lala”]



내장 객체 계층 구조

웹 브라우저가 제공하는 객체와 Web API

JavaScript에서 웹 페이지 · 주소 · 이동 기록 · 저장소 · 네트워크를 제어하는 도구

ECMAScript 내장 객체(Array · Math 등)와 달리, 브라우저 실행 환경이 제공

window: 현재 창/탭의 전역 환경

주요 객체로의 접근 (Access References, 상속 관계가 아님)

`window.document` `window.location` `window.history` `window.navigator` `window.screen`

브라우저의 일반 페이지 스크립트에서는 window 접두어를 생략해 사용 가능

1 window

- 창 크기 · 대화상자 · 타이머

```
window.innerWidth;
window.alert("안녕하세요");
setTimeout(() => alert("알림"), 1000);
```

타이머는 지정 시간 이후 실행을 예약

2 document · Element (DOM)

- HTML 요소 선택 · 변경 · 이벤트 처리

```
const el = document.querySelector("#msg");
el.textContent = "완료";
```

예제: #msg 요소가 존재한다고 가정

3 location

- 현재 URL 확인 · 페이지 이동

```
location.href;  // 현재 URL
location.assign("/home");
```

4 history

- 현재 탭의 방문 기록 이동

```
history.back();
history.pushState({}, "", "/page");
```

pushState: 같은 출처 URL, 페이지 로드 없이 기록 추가
(전체 방문 기록을 읽을 수는 없음)

5 navigator

- 브라우저 · 언어 정보와 기기 API 접근

```
navigator.language;  // 예: "ko-KR"
```

위치 · 클립보드 등은 권한/보안 조건 적용

6 screen

- 사용자 화면 정보

```
screen.width;
screen.height;
```

화면 크기 ≠ 페이지 표시 영역(window.innerWidth)

7 localStorage · sessionStorage

- 문자열 키-값 저장

```
localStorage.setItem("theme", "dark");
localStorage.getItem("theme");
```

- localStorage: 세션 이후에도 유지 가능
- sessionStorage: 출처 · 탭별 페이지 세션 동안 유지
- 출처(origin)별 저장

8 console

- 개발자 도구에 실행 정보 · 오류 출력

```
console.log("확인", 42);
console.error("오류");
```

사용자 화면 출력과 구분

9 네트워크 Web API

fetch 함수 · WebSocket 객체

- HTTP 요청 · 실시간 양방향 통신

```
fetch("/api/data")
  .then(r => r.json());
```

fetch는 함수: 응답 처리는 Promise로 연결
다른 출처 요청에는 CORS 동 적용

역할 구분: window(창) · document(DOM) · location(URL) · history(이동) · Storage(저장)

Web API는 브라우저 기능을 JavaScript로 사용할 수 있게 연결한다

참고: MDN Web APIs

- **window 객체**
  - 가장 상위의 객체로서 우리가 자바스크립트하는 모든 작업들이 이 window객체에서 이루어지게 된다.
  - window 하위의 객체를 가르칠 때 생략해서 나타냄.
    - ⇔ 예) window.document.write() 여기에서 window가 생략이 가능하다.
  - Properties, data, attribute, member data
    - ⇔ status : 상태선에 출력할 임의의 문자열을 지정하는 하나의 변수
    - ⇔ defaultStatus : status가 바뀔 때 까지 나타나는 상태선의 변수.
    - ⇔ self :자기 자신을 가리킴
      - ⇒ 예) window.document.write() = self.document.write()
    - ⇔ parent/top
    - ⇔ frames
      - ⇒ 프레임은 배열 - 윈도우에 포함되어 있는 프레임들을 가리킴
      - ⇒ frames[0]: 첫 번째 프레임 frames[1] 두 번째 프레임
      - ⇒"fr1"을 가리킬때는 frames["fr1"]으로 나타낸다
    - ⇔ opener : 새로 만들어진 윈도우의 문자열 출력이나 특성을 수정할 때 사용

- methods
  - ⇔ open() : 브라우저와 같은 모양의 윈도우 하나를 만들어 낸다.
    - ⇒ open("URL", "window name",["window feature list"])
    - ⇒ feature list : toolbar, location, directories, status, menubar, scrollbars, resizable, copyhistory, width, height
  - ⇔ close() : open() 메소드를 통해 만든 윈도우를 닫는 역할을 한다
  - ⇔ alert()
    - ⇒ 사용자에게 정보나 경고 메시지를 전하기 위한 간단한 다이얼 로그 박스
    - ⇒ 디버깅 할 때 유용 자바스크립트에서 특정 변수의 값을 알고 싶을 때
  - ⇔ confirm()
    - ⇒ 사용자에게로부터 대답을 받기 위한 다이얼로그 박스를 생성
    - ⇒ 형식 : confirm(message)
    - ⇒ 매개변수 : message (다이알로그 박스에 보여줄 메시지 문자열)
    - ⇒ 리턴값 : true (OK를 click), false (Cancel을 click)
  - ⇔ promt()
    - ⇒ confirm()과 같이 질문을 받되 문자열을 직접 입력 받을 수 있음.
    - ⇒ 형식 : prompt(message, [defaultAnswer])
    - ⇒ 매개변수 : message: 다이얼로그 박스에 보여줄 메시지 문자열 (질문)
      defaultAnswer: 대답을 입력할 박스에서 미리 보여줄 문자열
    - ⇒ 리턴값 : OK 버튼을 누른 경우: 입력 박스에 입력된 문자열
      Cancel 버튼을 누른 경우: null

⇔ setTimeout()

⇒ 지정된 시간동안 기다린 후 지정된 명령을 실행시키는 매우 독특한 기능

⇒ 주기적으로 시간을 체크해야 하는 프로그램이나, 잠시 안내 메시지를 보여 준 후에 본래의 홈페이지를 보여 준다는 등의 이용을 할수 있다.

⇒형식 : setTimeout(function, time)

⇒ 매개변수 : function: 지정된 시간이 지난 후 실행할 함수나 명령
time: 기다릴 시간(단위: ms, 1/1000초)

⇔ clearTimeout()

⇒ setTimeout()의 해제 시키는 역할

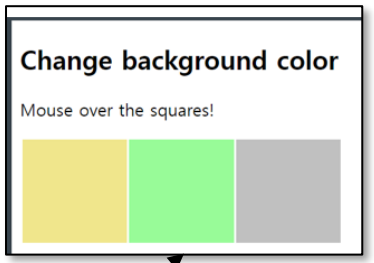
– Event Handler

⇔ onLoad(), onUnload(), onError(), onFocus(), onBlur()

- **document 객체**
  - HTML 문서의 <body> ... </body>와 직접적인 연관
  - properties
    - ⇔ title : 문서의 제목
    - ⇔ location : 문서의 URL 위치
    - ⇔ lastModified : 문서를 마지막으로 수정한 날짜
    - ⇔ referrer : 링크로 현재 문서에 왔을 때 이전 문서의 URL 위치
    - ⇔ bgColor : 문서의 배경색
    - ⇔ fgColor : 문서의 전경색
    - ⇔ linkColor : 문서에서 링크를 표시하는 색
    - ⇔ alinkColor : 링크를 클릭했을 때 나타나는 색
    - ⇔ vlinkColor : 이전에 방문했던 링크를 표시하는 색
    - ⇔ anchors : 문서에 있는 표식들의 배열
    - ⇔ forms : 문서에 있는 입력 양식들의 배열
    - ⇔ links : 문서에 있는 링크들의 배열
    - ⇔ cookies : 클라이언트 쪽의 PC에 저장된 정보
    - ⇔ images : 문서에 있는 이미지들의 배열
    - ⇔ applets : 문서에 있는 자바 애플릿들의 배열
    - ⇔ embeds : 문서에 있는 플러그인들의 배열

클릭 → 배경 색깔 변경

마우스 over → 배경 색깔 변경



https://www.w3schools.com/js/tryit.asp?filename=tryjs_dom_css_bgcolor 실습

- methods
  - ⇔ open() : 윈도우에서 데이터를 보낼 준비를 하거나 데이터를 모두 보냈다는 것을 알리기 위해 사용되는 것들이다.
  - ⇔ close() : open() 메소드로 열려진 문서를 닫아 주는 역할
  - ⇔ clear() : 브라우저에 나타난 문서를 지워 주는 열할
  - ⇔ write()/ writeln() : 지정된 문서의 데이터를 출력

- **history 객체**
  - 브라우저의 히스토리 리스트 정보를 저장해 두는 곳.
  - 가장 최근에 방문하였던 URL 주소들을 의미, 단 자바 스크립트를 사용하여 히스토리 리스트 속에 있는 URL 주소를 알아낼 수는 없다.
  - property
    - ⇔ lenth : 히스토리 리스트에 포함되어 있는 URL 주소들의 개수를 나타냄
  - methods
    - ⇔ back()/ forward() : 앞뒤로 오고 갈수 있도록 만든 것.
    - ⇔ go(num 혹은 URL) : 임의의 위치에 이동을 원 할 때.

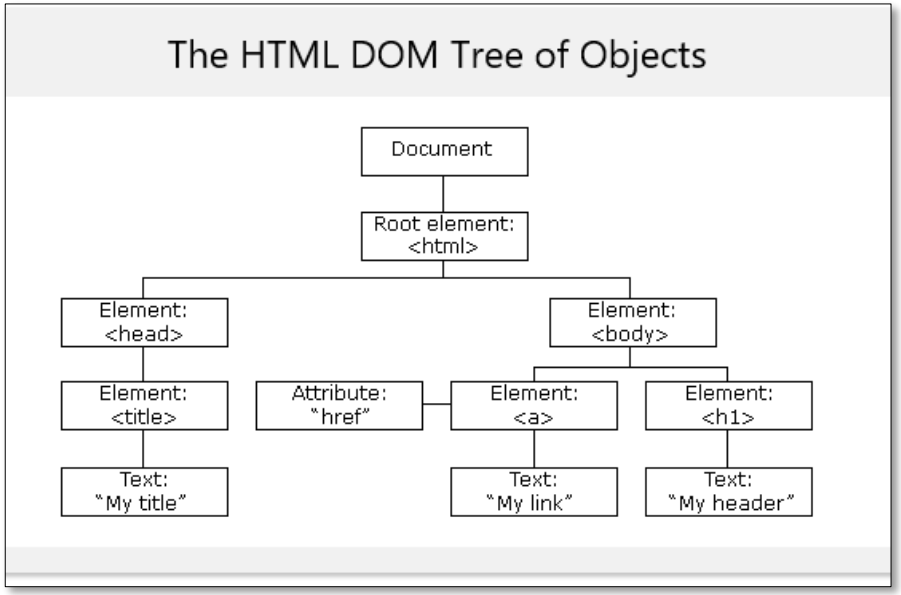
- **location 객체**
  - 현재 열려진 윈도우의 URL 주소에 관한 정보를 제공하는 객체
  - URL : http:// www.digital.co.kr:80/html/pc/my.html#oc1
    - ⇔ href, protocol, host, hostname, port, pathname, hash

7. JavaScript HTML DOM

- **The HTML DOM (Document Object Model)**
  - **When a web page is loaded, the browser creates a Document Object Model of the page.**

```
<html>
  <head>
     <title> My title </title>
  </head>

  <body>
     <a href="…"> My link </a>
     <h1> My header </h1>
  </body>
</html>
```



  - **With the object model, JavaScript gets all the power it needs to create dynamic HTML**
    - ⇔ **JavaScript can change all the HTML elements in the page, all the HTML attributes in the page, all the CSS styles in the page**
    - ⇔ **JavaScript can remove existing HTML elements and attributes**
    - ⇔ **JavaScript can add new HTML elements and attributes**
    - ⇔ **JavaScript can react to all existing HTML events in the page**
    - ⇔ **JavaScript can create new HTML events in the page**

DOM: 웹 문서를 객체로 다루는 방법

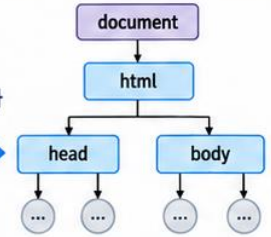
Document Object Model — HTML 구조를 메모리의 객체 트리로 표현하는 표준 인터페이스

① 왜 DOM이 필요한가?

```
<html>
 <head>...</head>
 <body>
  <p>...</p>
  ...
 </body>
</html>
```

HTML 문서 (텍스트 문서)

브라우저가 파싱



조작 가능한 DOM 노드 트리 (메모리의 객체)

HTML은 문서의 구조를 기술
제목, 단락, 목록 등 문서의 의미와 구조를 나타냄

DOM은 요소를 선택 · 읽기 · 변경할 객체와 API를 제공
JavaScript가 사용할 수 있는 인터페이스

JavaScript가 사용자 입력에 따라 내용을 동적으로 갱신
이벤트에 반응하여 DOM을 조작하고 화면을 변경

장바구니 수량 · 검색 결과 · 폼 오류 메시지 등 다양한 기능이 DOM 조작으로 구현됨

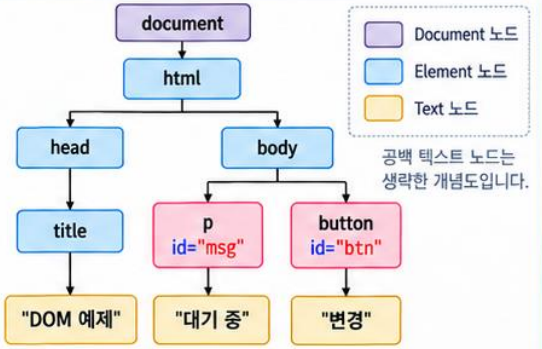
페이지 전체를 다시 받지 않고도 일부 화면 변경 가능
필요한 부분만 DOM을 조작하여 동적으로 업데이트할 수 있다.

② HTML → DOM 트리

HTML 예제 (텍스트 문서)

```
1  <html>
2    <head>
3      <title>DOM 예제</title>
4    </head>
5    <body>
6      <p id="msg">대기 중</p>
7      <button id="btn">변경</button>
8    </body>
9  </html>
```

DOM 트리 (메모리의 객체 구조)



공백 텍스트 노드는 생략한 개념도입니다.

③ 클릭 → JavaScript → DOM 변경

클릭 전 (브라우저 화면)



① 클릭 이벤트

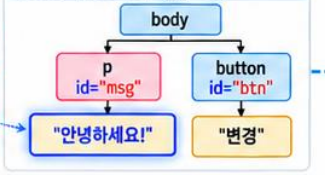
JavaScript 코드 (요소가 생성된 뒤 실행)

```
1  const msg = document.querySelector("#msg");
2  document.querySelector("#btn")
3    .addEventListener("click", () => {
4      msg.textContent = "안녕하세요!";
5      msg.style.color = "blue";
6    });
```

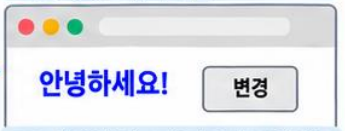
② 이벤트 핸들러 실행

③ 텍스트·스타일 변경
DOM 트리의 내용이 메모리에서 변경됨

DOM (메모리의 객체 트리)



클릭 후 (브라우저 화면)



DOM이 변경되면 브라우저가 이를 반영하여 화면이 업데이트됩니다.

④ DOM이 바뀌면 화면은 어떻게 바뀌나?

DOM 변경 (요소 · 속성 · 텍스트 수정)
→ **스타일 계산** (어떤 CSS 규칙이 적용되는지 계산)
← **CSS / CSSOM** (스타일 규칙)
→ **레이아웃** (필요 시) 요소의 크기와 위치 계산
→ **페인트 · 합성** (필요 시) 화면에 그리기(레스터화) 후 합성
→ **갱신된 화면** (브라우저에 표시)

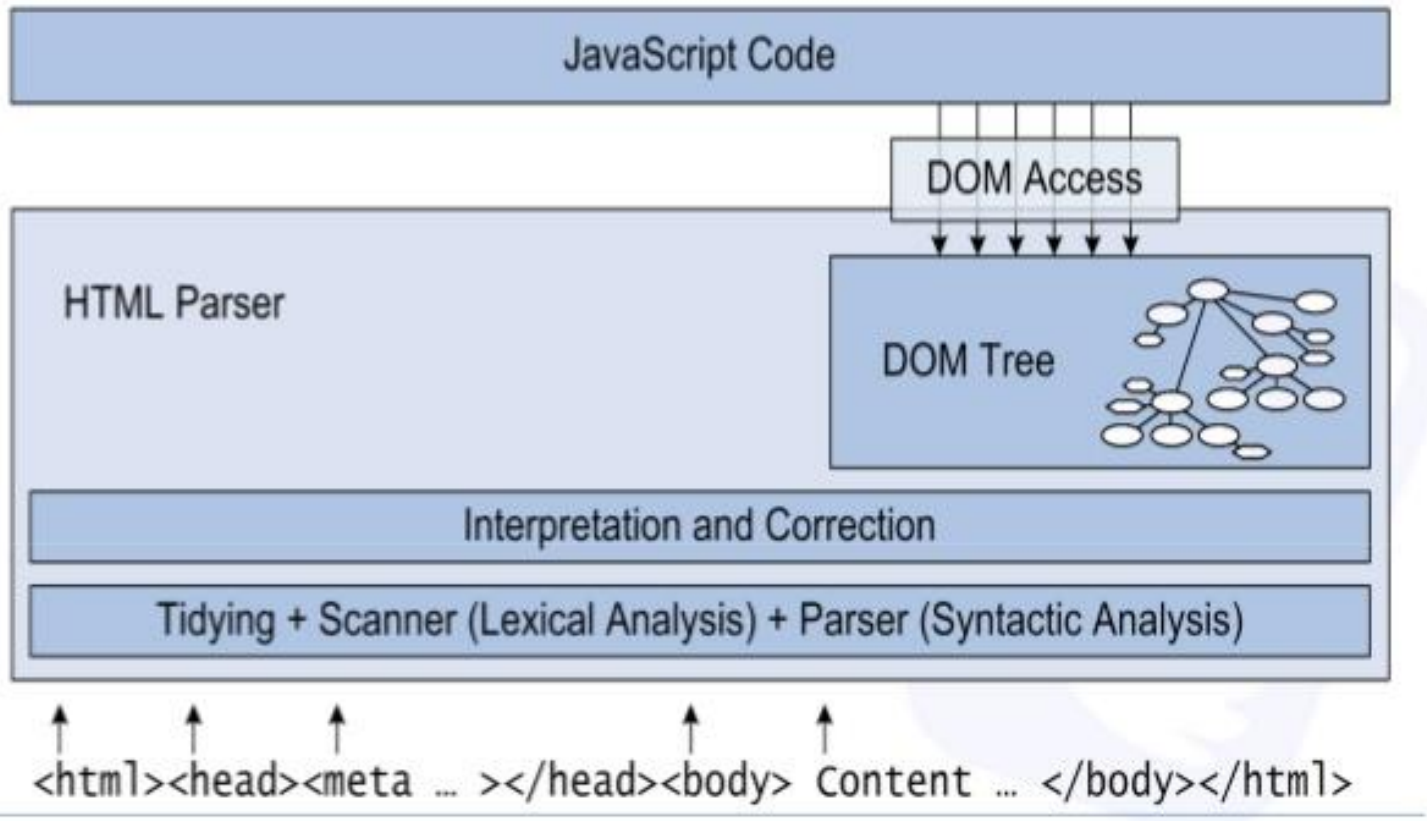
- **내용 · 크기 변화** : 배치 계산이 필요할 수 있음
- **색상 변화** : 보통 배치 변경 없이 다시 그리기

브라우저는 변경을 모아 렌더링 시점에 반영할 수 있음
모든 DOM 수정이 즉시 화면을 그리는 것은 아니며, 브라우저의 최적화에 따라 여러 변경이 함께 처리될 수 있습니다.

DOM 수정은 브라우저 메모리의 문서 변경이며, 서버의 원본 HTML 파일을 바꾸는 것은 아니다.

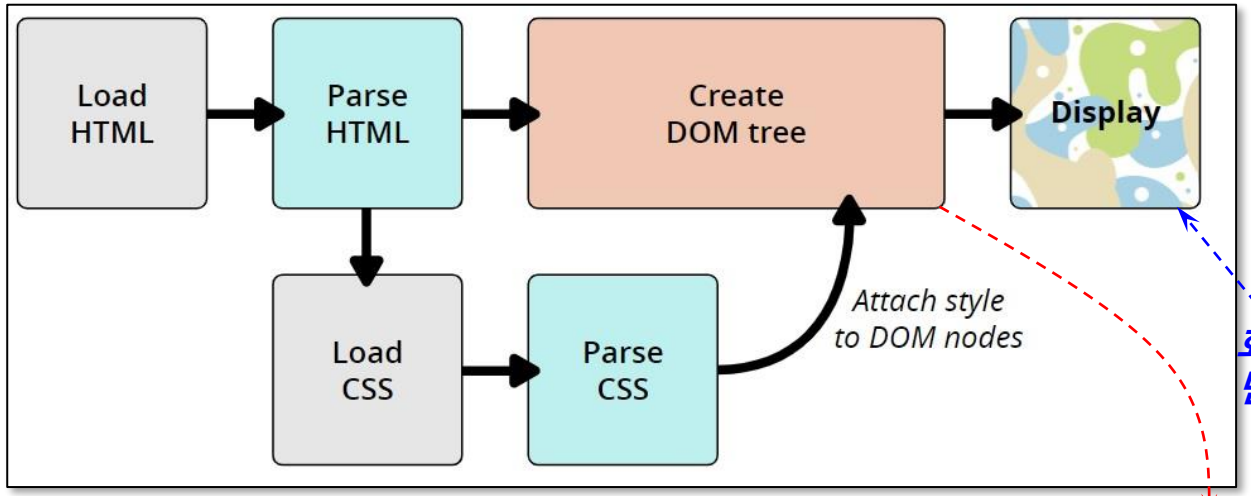
공식 참고 : MDN DOM · Rendering

Javascript : DOM control



32

PHP DOM vs. JavaScript DOM



JavaScript 프로그램

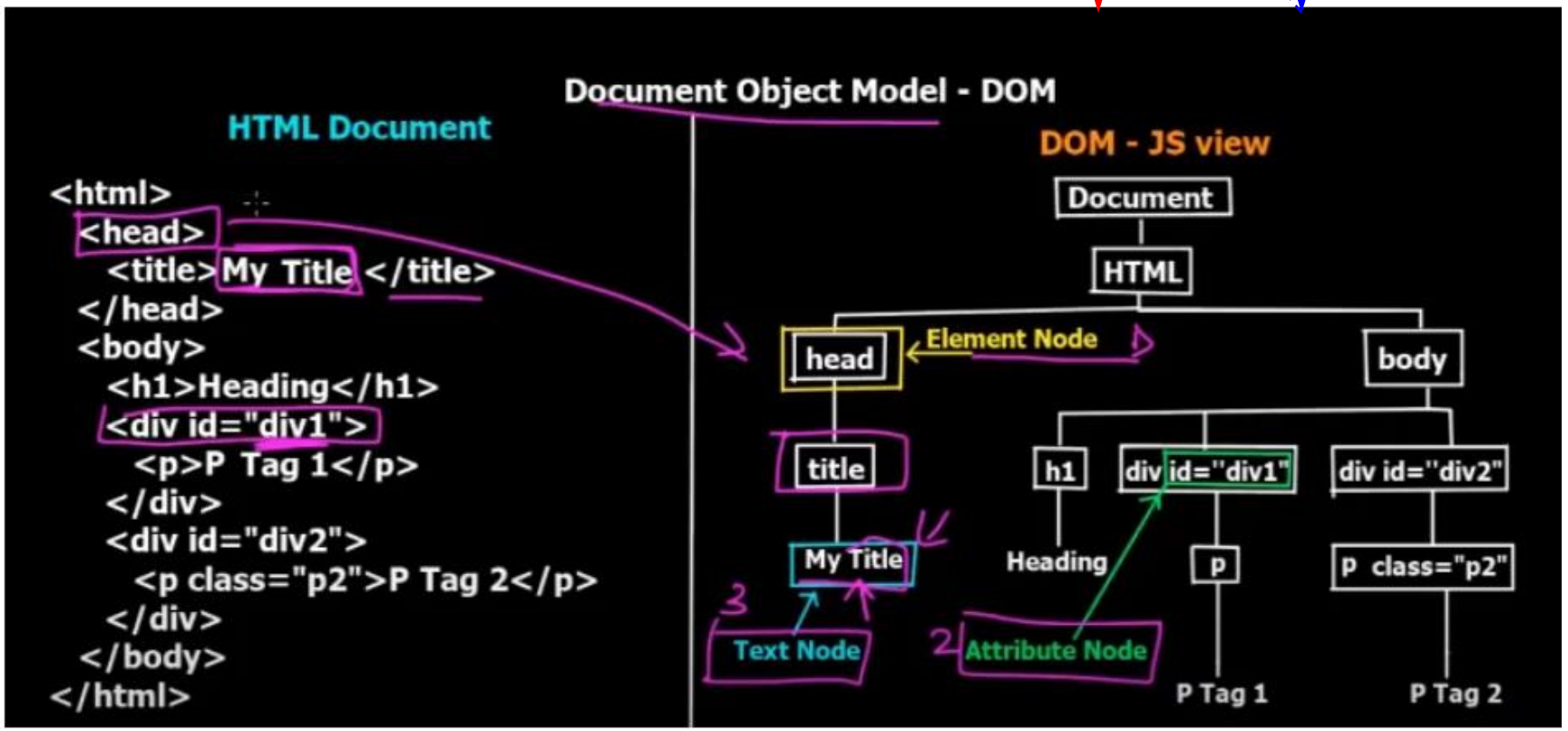
```
<script>
    let hour = 15;

    if (hour < 12) {
        console.log("좋은 아침입니다!");
    } else if (hour < 18) {
        console.log("즐거운 오후되세요!");
    } else {
        console.log("좋은 밤 되세요!");
    }
</script>
```

Events

화면 내용 변경

get, change, add, or delete HTML elements.

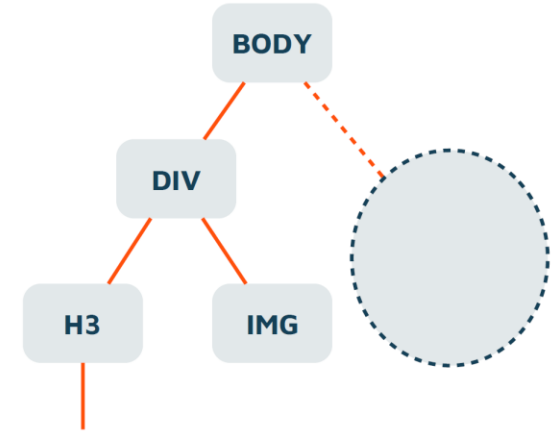


- **What is the DOM?**
  - The DOM is a W3C (World Wide Web Consortium) standard.
  - The DOM defines a standard for *accessing documents*:
    - ⇔ *"The W3C Document Object Model (DOM) is a platform and language-neutral interface that allows programs and scripts to dynamically access and update the ① content, ② structure, and ③ style of a document."*
  - The W3C DOM standard is separated into 3 different parts:
    - ⇔ Core DOM - standard model for all document types
    - ⇔ XML DOM - standard model for XML documents
    - ⇔ HTML DOM - standard model for HTML documents

① Contents
② Structure
③ Style

TRAVERSING THE DOM

```
var body = document.body;
var div = body.children[0];
var h3 = div.children[0];
var textNode = h3.childNodes[0];
var textString = textNode.nodeValue;
```




```
<BODY>
    <DIV>
      <H3> My First photo </H3>
       <IMG>    </IMG>
    <DIV>
    ……

</BODY>
```

- **What is the HTML DOM?**
  - The HTML DOM is a standard object model and programming interface for HTML.
  - It defines:
    - ⇔ The HTML elements as objects
    - ⇔ The properties of all HTML elements
    - ⇔ The methods to access all HTML elements
    - ⇔ The events for all HTML elements
  - In other words: The HTML DOM is a standard for ***how to get, change, add, or delete HTML elements.***

properties for traversing the DOM tree

`element.childNodes`		`element.children`
`element.parentNode`	**vs.**	`element.parentElement`
`element.previousSibling`		`element.previousElementSibling`
`element.nextSibling`		`element.nextElementSibling`

A **`Node`** can be anything in the DOM (Text, Comments, etc.), while **`Elements`** are the nodes that represent HTML elements

- **HTML DOM Methods**
  - ⇔ HTML DOM methods are actions you can perform (on HTML Elements).
  - ⇔ HTML DOM properties are values (of HTML Elements) that you can set or change.
  - **The DOM Programming Interface**
    - ⇔ The HTML DOM can be accessed with JavaScript (and with other programming languages).
    - ⇔ In the DOM, all HTML elements are defined as objects.
    - ⇔ The programming interface is the properties and methods of each object.
      - ⇒ A property is a value that you can get or set (like changing the content of an HTML element).
      - ⇒ A method is an action you can do (like add or deleting an HTML element).

```
<html>
<body>

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = "Hello World!";
</script>

</body>
</html>
```

method

property

이 method의 return 값은 element



http://www.w3schools.com/js/js_htmldom_methods.asp

- **HTML DOM Document**
  - **HTML DOM document Object (*built-in, pre-defined*)**
    - ⇔ **The document object represents your web page.**
    - ⇔ **If you want to access any element in an HTML page, you always start with accessing the document object.**

1) Finding HTML Elements

Method	Description
document.getElementById(*id*)	Find an element by element id
document.getElementsByTagName(*name*)	Find elements by tag name
document.getElementsByClassName(*name*)	Find elements by class name

2) Changing HTML Elements

Method	Description
element.innerHTML = *new html content*	Change the inner HTML of an element
element.attribute = new value	Change the attribute value of an HTML element
element.setAttribute(*attribute, value*)	Change the attribute value of an HTML element
element.style.*property = new style*	Change the style of an HTML element

3) Adding and Deleting Elements

Method	Description
document.createElement(*element*)	Create an HTML element
document.removeChild(*element*)	Remove an HTML element
document.appendChild(*element*)	Add an HTML element
document.replaceChild(*element*)	Replace an HTML element
document.write(*text*)	Write into the HTML output stream

4) Adding Events Handlers

Method	Description
document.getElementById(*id*).onclick = function(){*code*}	Adding event handler code to an onclick event

5) Finding HTML Element Objects

Property	Description	DOM
document.anchors	Returns all <a> elements that have a name attribute	1
document.applets	Returns all <applet> elements (Deprecated in HTML5)	1
document.baseURI	Returns the absolute base URI of the document	3
document.body	Returns the <body> element	1
document.cookie	Returns the document's cookie	1
document.doctype	Returns the document's doctype	3
document.documentElement	Returns the <html> element	3
document.documentMode	Returns the mode used by the browser	3
document.documentURI	Returns the URI of the document	3
document.domain	Returns the domain name of the document server	1
document.domConfig	Obsolete. Returns the DOM configuration	3
document.embeds	Returns all <embed> elements	3
document.forms	Returns all <form> elements	1
document.head	Returns the <head> element	3
document.images	Returns all <img> elements	1
document.implementation	Returns the DOM implementation	3
document.inputEncoding	Returns the document's encoding (character set)	3
document.lastModified	Returns the date and time the document was updated	3
document.links	Returns all <area> and <a> elements that have a href attribute	1
document.readyState	Returns the (loading) status of the document	3
document.referrer	Returns the URI of the referrer (the linking document)	1
document.scripts	Returns all <script> elements	3
document.strictErrorChecking	Returns if error checking is enforced	3
document.title	Returns the <title> element	1
document.URL	Returns the complete URL of the document	1

document.anchors[0].

HTML 문서가 브라우져에 load되면 built-in 객체인 document 객체의 property에 HTML 문서에 있는 element값들이 저장된다.

– 예제 : Finding HTML Elements

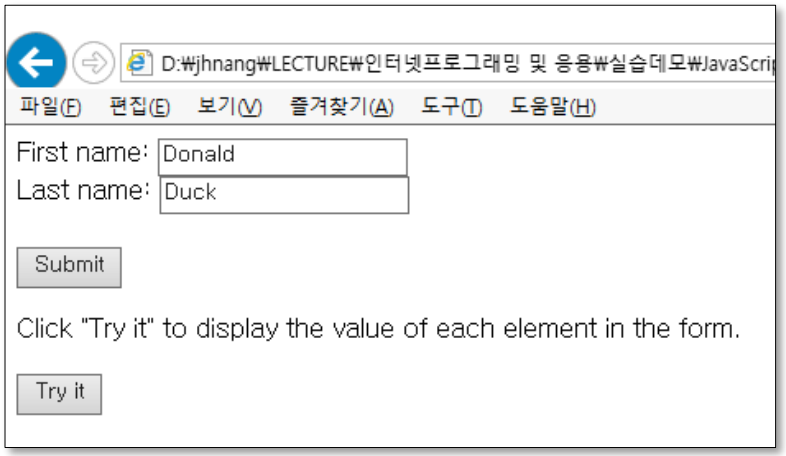
DOM_document.html

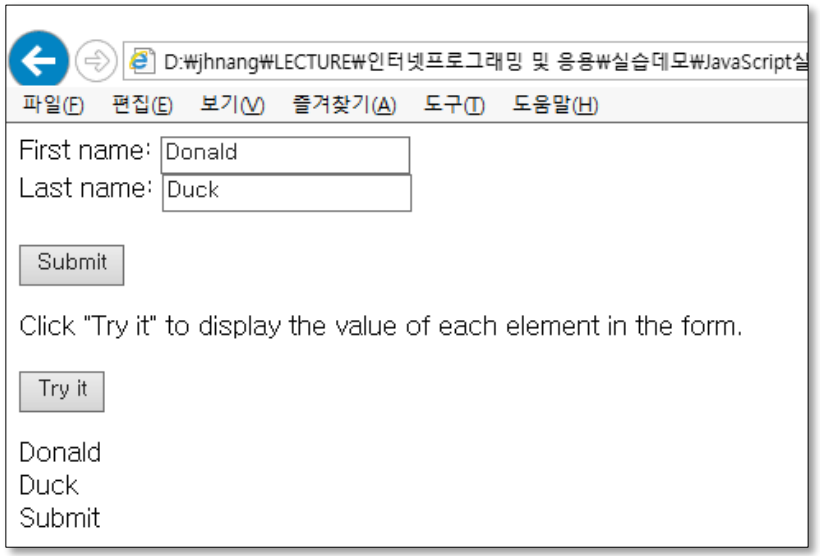
DOM/DOM_document.html

```
1  <!DOCTYPE html>
2  <html>
3  <body>
4
5  <form id="frm1" action="form_action.asp">
6    First name: <input type="text" name="fname" value="Donald"><br>
7    Last name: <input type="text" name="lname" value="Duck"><br><br>
8    <input type="submit" value="Submit">
9  </form>
10
11 <p>Click "Try it" to display the value of each element in the form.</p>
12
13 <button onclick="myFunction()">Try it</button>
14
15 <p id="demo"></p>
16
17 <script>
18 function myFunction() {
19     var x = document.forms["frm1"];
20     var text = "";
21     var i;
22     for (i = 0; i < x.length ;i++) {
23         text += x.elements[i].value + "<br>";
24     }
25     document.getElementById("demo").innerHTML = text;
26 }
27 </script>
28
29 </body>
30 </html>
```

associative array

DOM_document.html





– 예제 : Changing HTML Style

```html
<!DOCTYPE html>
<html>
<body>

<h1 id="id1">My Heading 1</h1>

<button type="button"
onclick="document.getElementById('id1').style.color = 'red'">
Click Me!</button>

</body>
</html>
```





– 예제 : Animation

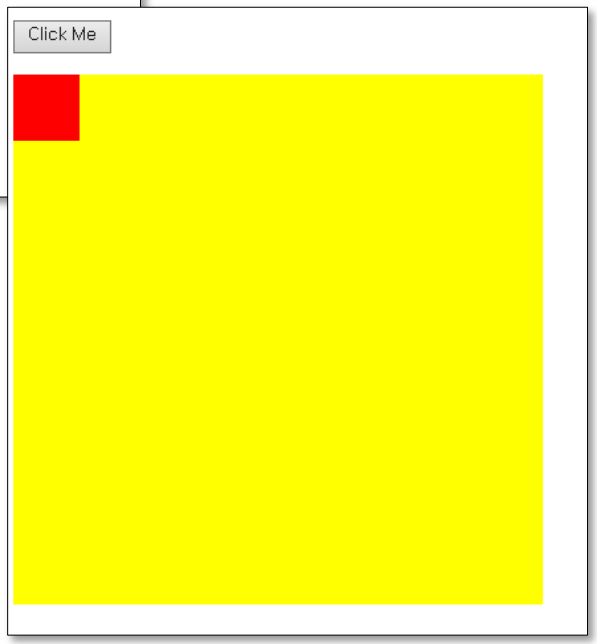
- nested function definition

```
1  <!DOCTYPE html>
2  <html>
3  <style>
4  #container {
5    width: 400px;
6    height: 400px;
7    position: relative;
8    background: yellow;
9  }
10 #animate {
11   width: 50px;
12   height: 50px;
13   position: absolute;
14   background-color: red;
15 }
16 </style>
17 <body>
18
19 <p>
20 <button onclick="myMove()">Click Me</button>
21 </p>
22
23 <div id ="container">
24 <div id ="animate"></div>
25 </div>
```

```
27 <script>
28 function myMove() {
29   var elem = document.getElementById("animate");
30   var pos = 0;
31   var id = setInterval(frame, 5);
32   function frame() {
33     if (pos == 350) {
34       clearInterval(id);
35     } else {
36       pos++;
37       elem.style.top = pos + 'px';
38       elem.style.left = pos + 'px';
39     }
40   }
41 }
42 </script>
43
44 </body>
45 </html>
46
```

static scoping rule

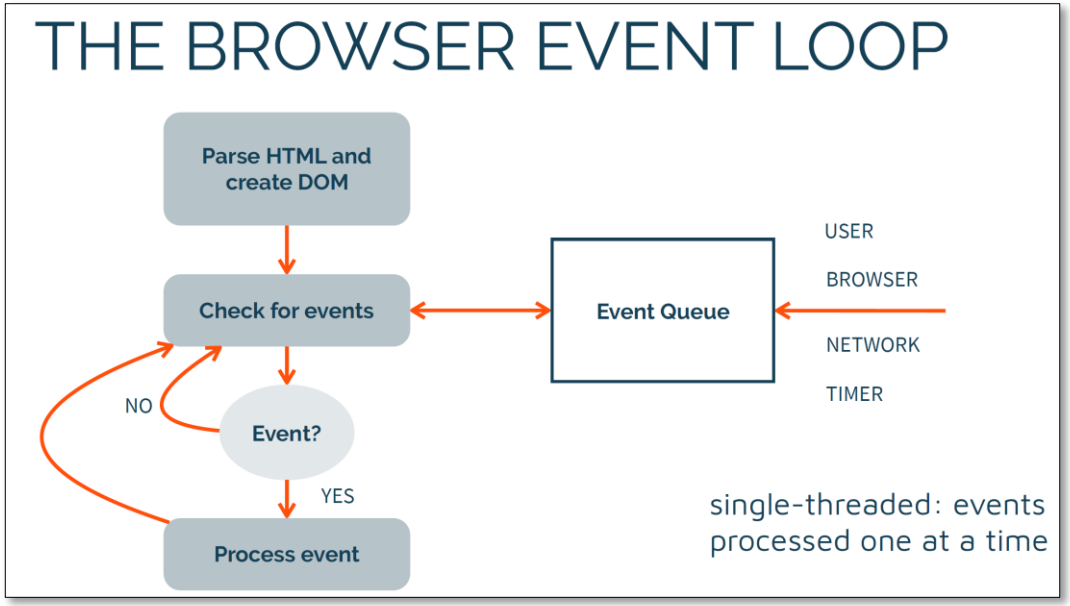
animation.html

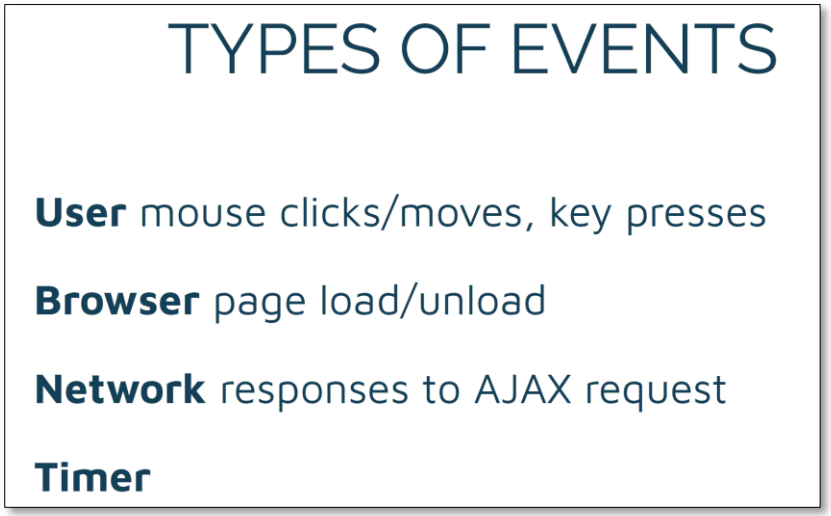


- 4배속으로 움직이게 수정 ~~
- 계속 움직이게 수정 ~~

- **HTML DOM Events**
  - **Reacting to Events**

⇔ **A JavaScript can be executed when an event occurs, like when a user clicks on an HTML element.**






```html
<!DOCTYPE html>
<html>
<body>

<h1 onclick="this.innerHTML='Ooops!'">Click on this text!</h1>

</body>
</html>
```

```html
<!DOCTYPE html>
<html>
<body>

<h1 onclick="changeText(this)">Click on this text!</h1>

<script>
function changeText(id) {
    id.innerHTML = "Ooops!";
}
</script>

</body>
</html>
```

HTML DOM 이벤트 처리: Event Loop와 Event Type

이벤트 등록 → 발생 → 핸들러 실행 → DOM 변경 → 화면 갱신

1 이벤트 루프: 언제 핸들러를 실행할까?



1. 메인 스레드의 현재 JS 실행을 끝낸 뒤 다음 Task 처리
2. Task 후 Microtask 처리, 렌더링은 매번 발생하지 않음
3. 긴 핸들러는 입력 반응과 화면 갱신을 지연

dispatchEvent()로 보낸 이벤트는 호출 중 동기 처리(별도의 Task 대기열에 넣지 않음).

2 주요 이벤트 종류

대상/기능	Event type	발생 시점
마우스 · 포인터	click · pointerdown	클릭 · 포인터 누름
키보드	keydown · keyup	키 누름 · 해제
입력 · 폼	input · change · submit	값 편집 · 변경 확정 · 제출
포커스	focus · blur	포커스 획득 · 해제
문서 · 리소스	DOMContentLoaded · load	DOM 준비 · 리소스 로드
화면 · 스크롤	resize · scroll	창 크기 · 스크롤 변화

- DOMContentLoaded: HTML 파싱 및 지연 스크립트 실행 완료
- load: 이미지 등 주요 리소스 로드 완료

이벤트별 발생 대상과 버블링 여부가 다름.

3 등록과 실행 예제



HTML + JavaScript — #btn · #msg 요소가 생성된 뒤 실행

```
const btn = document.querySelector("#btn");
btn.addEventListener("click", (event) => {
   document.querySelector("#msg")
      .textContent = "클릭했습니다!";
   console.log(event.type); // "click"
});
```

type: 이벤트 종류 (예: "click")

target: 이벤트가 발생한 대상 (예: 클릭한 버튼 요소)

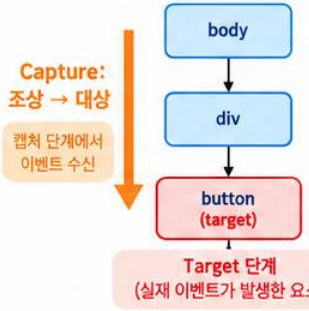
currentTarget: 현재 리스너의 대상 (여기서는 #btn)



addEventListener는 핸들러를 등록하며, 즉시 실행하지 않음.

이벤트의 종류 이름은 "click"이며, "onclick"이 아님.

4 DOM 안에서의 이벤트 전파



캡처 리스너 (조상 단계에서 실행)

```
div.addEventListener("click", handler, {
  capture: true
});
```

기본 리스너 (버블 단계에서 실행)

```
div.addEventListener("click", handler);
// 대상에서도 실행됨
```

버블링하지 않는 이벤트도 있음 (예: focus).

stopPropagation(): 전파 중단
preventDefault(): 취소 가능한 기본 동작 취소.

Event Loop는 실행 시점, 이벤트 전파는 DOM 내 전달 경로를 설명한다.

참고: MDN Events · Microtasks

HTML 파일 (연습문제1_post.html)

```
<!DOCTYPE html> <html> <body>
<h2>Finding HTML Elements by Tag Name</h2>

<p>Hello World!</p>

<p>This example demonstrates the <b>getElementsByTagName</b> method.</p>

<p id="demo"></p>

<script>
var x = document.getElementsByTagName("p");
document.getElementById('demo').innerHTML=  'The text in first paragraph (index 0) is: ' + /*여기를 채워라 */
</script> </body> </html>
```

출력결과

Finding HTML Elements by Tag Name

Hello World!

This example demonstrates the **getElementsByTagName** method.

The text in first paragraph (index 0) is: Hello World!

HTML DOM element Properties and Methods
➔ https://www.w3schools.com/jsref/dom_obj_all.asp

```
x[0].innerHTML;
```

```html
<!DOCTYPE html>
<html>
<body>

<p>Click "Try it" to execute the displayDate() function.</p>

<button id="myBtn">Try it</button>

<p id="demo"></p>

<script>
document.getElementById("myBtn").onclick = displayDate;

function displayDate() {
    document.getElementById("demo").innerHTML = Date();
}
</script>

</body>
</html>
```

Event handler 등록

Click "Try it" to execute the displayDate() function.

Try it

Wed Nov 02 2016 19:41:07 GMT+0900 (대한민국 표준시)

- **HTML DOM EventListener**
  - **addEventListener() method**
    - ⇔ *element*.addEventListener(*event, function, useCapture*);
      - ⇒ The first parameter is the type of the event (like "click" or "mousedown").
      - ⇒ The second parameter is the function we want to call when the event occurs.
      - ⇒ The third parameter is a boolean value specifying whether to use event bubbling or event capturing. This parameter is optional.
        - → *bubbling* the inner most element's event is handled first and then the outer (false, default)
        - → *capturing* the outer most element's event is handled first and then the inner (ture)
    - ⇔ attaches an event handler to the specified element.
    - ⇔ attaches an event handler to an element without overwriting existing event handlers.
  - **removeEventListener() method**
    - ⇔ *element*.removeEventListener("mousemove", myFunction);

Browser Support

The numbers in the table specifies the first browser version that fully supports these methods.

Method					
addEventListener()	1.0	9.0	1.0	1.0	7.0
removeEventListener()	1.0	9.0	1.0	1.0	7.0

– Example

```
<!DOCTYPE html>
<html>
<body>

<p>This example uses the addEventListener() method to attach a click event to a
button.</p>

<button id="myBtn">Try it</button>

<p id="demo"></p>

<script>
document.getElementById("myBtn").addEventListener("click", displayDate);

function displayDate() {
    document.getElementById("demo").innerHTML = Date();
}
</script>

</body>
</html>
```

This example uses the addEventListener() method to attach a click event to a button.

Try it

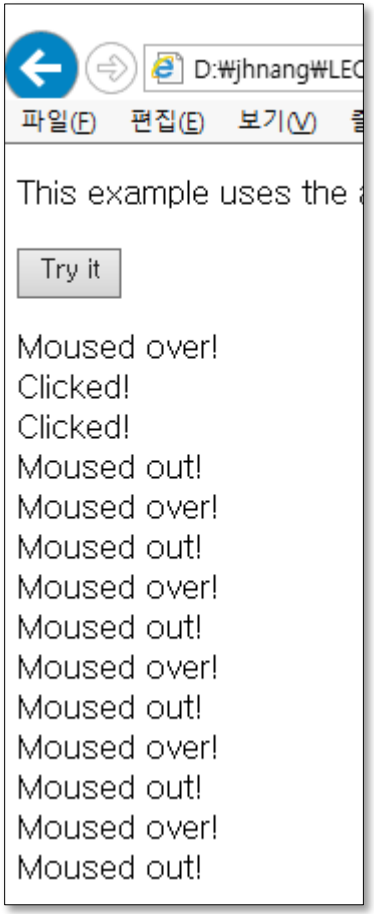
Wed Nov 02 2016 19:43:25 GMT+0900 (대한민국 표준시)

– Example

eventhandler.html

```
1  <!DOCTYPE html>
2  <html>
3  <body>
4
5  <p>This example uses the addEventListener() method to add many events on the same button.</p>
6
7  <button id="myBtn">Try it</button>
8
9  <p id="demo"></p>
10
11 <script>
12 var x = document.getElementById("myBtn");
13 x.addEventListener("mouseover", myFunction);
14 x.addEventListener("click", mySecondFunction);
15 x.addEventListener("mouseout", myThirdFunction);
16
17 function myFunction() {
18     document.getElementById("demo").innerHTML += "Moused over!<br>";
19 }
20
21 function mySecondFunction() {
22     document.getElementById("demo").innerHTML += "Clicked!<br>";
23 }
24
25 function myThirdFunction() {
26     document.getElementById("demo").innerHTML += "Moused out!<br>";
27 }
28 </script>
29
30 </body>
31 </html>
32
```

msg concatenation



실습 : eventhandler4_URL_OPEN – 문제.html
dblclick (ondblclick) :
마우스 버튼을 두 번 연속 더블 클릭 함
① **Msg Rest**
② **특정 사이트로 이동**

eventhandler4_URL_OPEN_정답.html

HTML DOM Navigation: 부모·자식·형제 노드 탐색

선택한 노드를 기준으로 DOM 트리의 관계를 따라 이동한다

① 예제 HTML과 DOM 트리

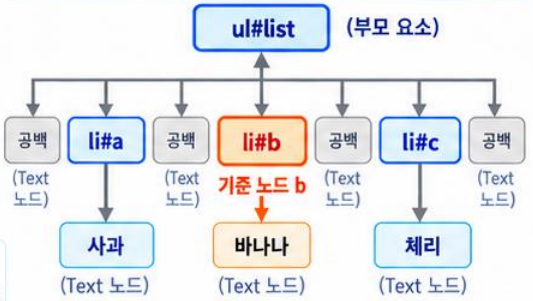
예제 HTML

```
<ul id="list">
  <li id="a">사과</li>
  <li id="b">바나나</li>
  <li id="c">체리</li>
</ul>
```

노드 참조 설정 코드

```
const b = document.getElementById('b');
const list = document.getElementById('list');
```

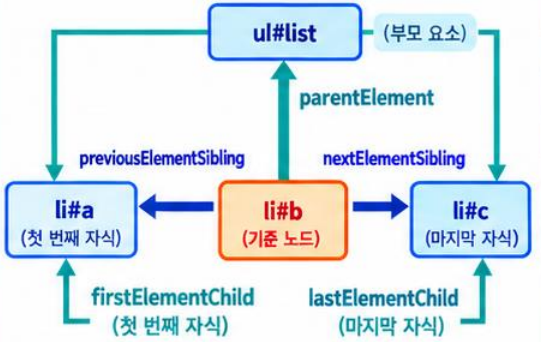
DOM 트리 구조



! 줄바꿈 · 들여쓰기도 Text 노드가 될 수 있음 (공백 Text 노드)

② 요소 관계로 이동 (Element 탐색)

요소 탐색 관계도 (기준 노드: li#b)



탐색 속성과 결과

b.parentElement	ul#list
b.previousElementSibling	li#a
b.nextElementSibling	li#c
list.firstElementChild	li#a
list.lastElementChild	li#c
list.children	3개 li 요소

! children은 직계 자식 요소의 HTMLCollection입니다.

③ Node 탐색 vs Element 탐색

관계	모든 노드 (Node 탐색)	요소만 (Element 탐색)
부모	parentNode	parentElement
자식 목록	childNodes	children
첫 · 마지막 자식	firstChild / lastChild	firstElementChild / lastElementChild
이전 · 다음 형제	previousSibling / nextSibling	previousElementSibling / nextElementSibling

예제에서의 결과 비교

list.firstChild	공백 Text 노드 (줄바꿈 · 들여쓰기)	b.nextSibling	공백 Text 노드 (줄바꿈 · 들여쓰기)
list.firstElementChild	li#a	b.nextElementSibling	li#c

- childNodes: NodeList (텍스트 노드, 주석 노드 등 모든 노드 포함)
- children / childNodes는 **실시간(live) 컬렉션**입니다.
- 대상이 없으면 단일 탐색 속성은 null을 반환합니다. (단, children / childNodes는 빈 컬렉션을 반환)

④ 탐색 후 읽기·변경

탐색 결과를 이용한 읽기와 변경

```
b.parentElement.id;                          // "list"   (부모 ul의 id)
b.firstChild.nodeValue;                      // "바나나"  (li의 텍스트 노드 값)
b.nextElementSibling.textContent;            // "체리"   (다음 형제 li의 텍스트 내용)
b.nextElementSibling.textContent = "포도";   // 다음 형제 li의 내용 변경
```

브라우저에서의 결과 (변경 전 → 변경 후)

변경 전	변경 후
• 사과	• 사과
• 바나나	• 바나나
• 체리	• 포도

! 탐색으로 얻은 것은 같은 DOM 노드의 참조이므로, 내용을 변경하면 실제 문서의 해당 요소가 바뀝니다.

추가 탐색 방법

- querySelector() : CSS 선택자로 찾기
  `const item = document.querySelector('#c');`
  (CSS 선택자로 요소를 찾아 직접 참조할 수 있음)
- closest('ul'): 자신부터 조상 방향으로 조건 검색
  `b.closest('ul');`
  (b 자신부터 위로 올라가며 가장 가까운 ul 찾기)

요소만 다룰 때는 Element 탐색 속성으로 공백 Text 노드를 건너뛴다 | ! DOM Navigation은 페이지 이동(location/history)과 다른 개념입니다. 참고: MDN - DOM / Node / Element

– **DOM Nodes : According to the W3C HTML DOM standard, everything in an HTML document is a node:**

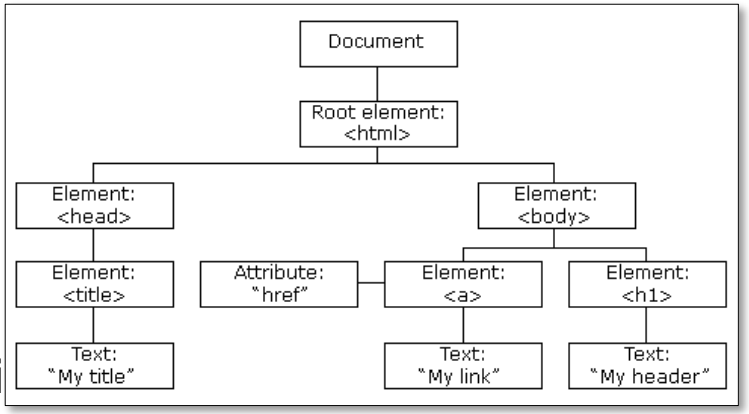
⇔ **The entire document is a document node**

⇔ **Every HTML element is an element node**

⇔ **text inside HTML elements are text nodes**

⇔ **Every HTML attribute is an attribute node**

⇔ **All comments are comment nodes**



– **Node Relationships**

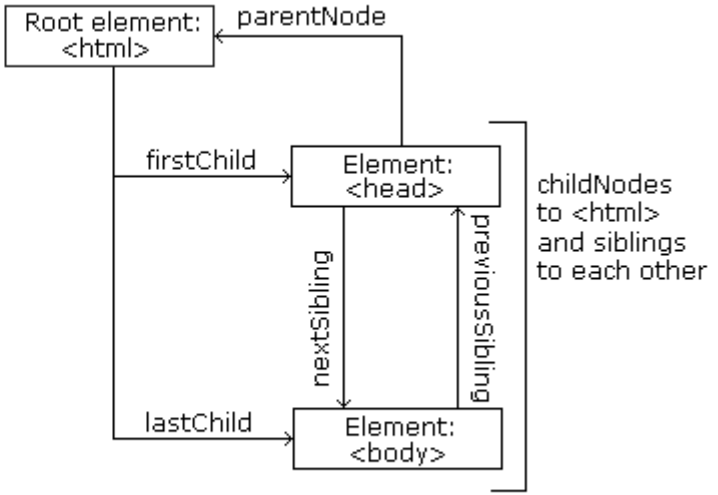
⇔ **The nodes in the node tree have a hierarchi**

```
<html>

  <head>
      <title>DOM Tutorial</title>
  </head>

  <body>
      <h1>DOM Lesson one</h1>
      <p>Hello world!</p>
  </body>

</html>
```



– Navigating Between Nodes

⇔ You can use the following node properties to navigate between nodes with JavaScript:

⇒ **parentNode**

⇒ **childNodes[*nodenumber*]**

⇒ **firstChild**

⇒ **lastChild**

⇒ **nextSibling**

⇒ **previousSibling**

```
<html>
<body>

<h1 id="intro">My First Page</h1>

<p id="demo">Hello!</p>

<script>
var myText = document.getElementById("intro").childNodes[0].nodeValue;
document.getElementById("demo").innerHTML = myText;
</script>

</body>
</html>
```

My First Page

My First Page

• HTML DOM Elements (Nodes)

Creating New HTML Elements (Nodes)

```
<div id="div1">
<p id="p1">This is a paragraph.</p>
<p id="p2">This is another paragraph.</p>
</div>

<script>
var para = document.createElement("p");
var node = document.createTextNode("This is new.");
para.appendChild(node);

var element = document.getElementById("div1");
element.appendChild(para);
</script>
```

This is a paragraph.

This is another paragraph.

This is new.

Removing Existing HTML Elements

```
<div id="div1">
<p id="p1">This is a paragraph.</p>
<p id="p2">This is another paragraph.</p>
</div>

<script>
var parent = document.getElementById("div1");
var child = document.getElementById("p1");
parent.removeChild(child);
</script>
```

This is another paragraph.

HTML DOM: nodeValue vs textContent

nodeValue는 노드 자체의 값, textContent는 요소 내부의 텍스트

① 위 그림의 예제에서는 결과가 같지만, 접근 대상이 다르다

예제 HTML

```
<li id="b">바나나</li>

const b = document.getElementById("b");
```

DOM 트리 구조 (노드 b)



주요 표현식과 결과

표현식	결과	설명
b.nodeValue	null	요소 노드의 nodeValue는 null
b.firstChild.nodeValue	"바나나"	텍스트 노드의 nodeValue는 실제 문자열
b.textContent	"바나나"	요소 내부의 텍스트(후손의 텍스트)
b.firstChild.textContent	"바나나"	텍스트 노드의 textContent도 동일한 문자열

! Element의 nodeValue는 null, Text의 nodeValue는 실제 문자열이다.

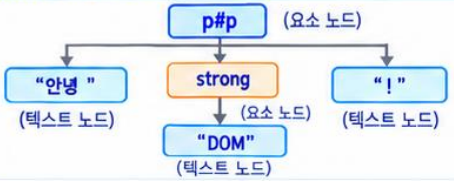
② nodeValue: 현재 노드 자체의 값

예제 HTML (중첩 구조)

```
<p id="p">안녕 <strong>DOM</strong>!</p>

const p = document.getElementById("p");
```

DOM 트리 구조 (노드 p)



nodeValue 예시

```
p.nodeValue;              // null (요소 노드)
p.firstChild.nodeValue;   // "안녕 " (첫 번째 텍스트 노드)
```

✓ nodeValue는 현재 노드 자신의 값만 반환한다.
위 예제에서는 첫 번째 텍스트 노드("안녕 ")의 값만 가져온다.

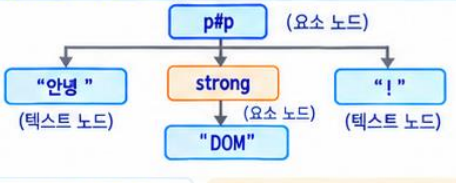
② textContent: 요소에서는 후손 텍스트를 이어 붙임

예제 HTML (중첩 구조)

```
<p id="p">안녕 <strong>DOM</strong>!</p>

const p = document.getElementById("p");
```

DOM 트리 구조 (노드 p)



textContent 예시

```
p.textContent;                          // "안녕 DOM!" (요소 내부의 전체 텍스트)
p.querySelector("strong").textContent;  // "DOM" (strong 요소 내부의 텍스트)
```

! 요소의 textContent에는 주석 내용은 포함되지 않음.

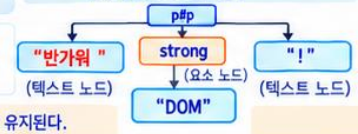
✓ textContent는 요소의 모든 후손 텍스트를 순서대로 이어 붙여 반환한다.
HTML 태그는 포함되지 않고, 텍스트만 추출된다.

③ 값을 변경하면? (각 예제는 원본 DOM에서 별도로 실행합니다.)

① 특정 Text 노드만 변경 (nodeValue)

```
p.firstChild.nodeValue = "반가워 ";
```

변경 후 DOM 트리 구조



브라우저의 출력 결과: 반가워 **DOM!**

✓ 해당 Text 노드만 변경: strong 요소는 그대로 유지된다.

② 요소의 전체 텍스트 변경 (textContent)

```
p.textContent = "반가워 DOM!";
```

변경 후 DOM 트리 구조



브라우저의 출력 결과: 반가워 DOM!

✓ 기존 자식 노드를 제거하고 하나의 Text 노드로 대체된다: strong 요소가 제거된다.

! p.nodeValue에 값을 대입해도 요소에는 효과 없음 (요소 노드의 nodeValue는 null로 유지됨).

✓ 요소의 전체 텍스트 처리: textContent / 특정 Text 노드만 처리: nodeValue

! textContent는 HTML 태그를 해석하지 않음. firstChild가 Text인지 확인해야 함.

참고: MDN - Node.nodeValue / Node.textContent

- **HTML DOM Node List**
  - **HTML DOM Node List**

⇔ **The getElementsByTagName() method returns *a node list*. A node list is an array-like collection of nodes.**

nodelist_bgclor_change.html

```
<!DOCTYPE html>
<html>
<body>

<p>This is a p element</p>

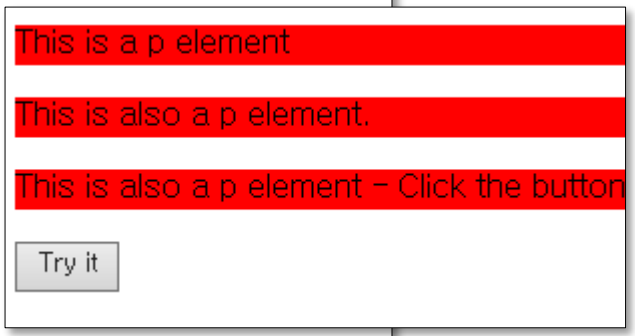
<p>This is also a p element.</p>

<p>This is also a p element - Click the button to change the background color of all p elements in this document.</p>

<button onclick="myFunction()">Try it</button>

<script>
function myFunction() {
    var myNodelist = document.getElementsByTagName("p");
    var i;
    for (i = 0; i < myNodelist.length; i++) {
        myNodelist[i].style.backgroundColor = "red";
    }
}
</script>

</body>
</html>
```



- 실습 : font를 바꾸는 연습….
➔ Googling "javascript DOM element font properties"

https://www.w3schools.com/jsref/prop_style_font.asp
https://www.w3schools.com/jsref/tryit.asp?filename=tryjsref_style_fontfamily2
https://www.w3schools.com/jsref/tryit.asp?filename=tryjsref_style_fontsize2

nodelist_font_change.html

8. JavaScript Browser BOM

- Window - The Browser Object Model (BOM)
  - allows JavaScript to "talk to" the browser.
- window Object
  - All global JavaScript objects, functions, and variables automatically become members of the window object.
  - Global variables are properties of the window object.
  - Global functions are methods of the window object.
  - Window Methods
    - ⇔ window.open() - open a new window
    - ⇔ window.close() - close the current window
    - ⇔ window.moveTo() -move the current window
    - ⇔ window.resizeTo() -resize the current window
- window.location object
  - window.location.href returns the href (URL) of the current page
  - window.location.hostname returns the domain name of the web host
  - window.location.pathname returns the path and filename of the current page
  - window.location.protocol returns the web protocol used (http:// or https://)

```
document.getElementById("demo").innerHTML =
"Page location is " + window.location.href;
```

```
Page location is http://www.w3schools.com/js/js_window_location.asp
```

- **Window history Object**
  - The window.history object contains the browsers history
  - Method
    - ⇔ history.back() - same as clicking back in the browser
    - ⇔ history.forward() - same as clicking forward in the browser

```
<html>
<head>
<script>
function goBack() {
    window.history.back()
}
</script>
</head>
<body>

<input type="button" value="Back" onclick="goBack()">

</body>
</html>
```

- **Window navigator Object**
  - The window.navigator object contains information about the visitor's browser.

Navigator Cookie Enabled

```
<p id="demo"></p>
<script>
document.getElementById("demo").innerHTML =
"Cookies Enabled is " + navigator.cookieEnabled;
</script>
```

Is Java enabled?

Try it

Java enabled is true

Did you know?

IE11, Chrome, Firefox, and Safari return appName "Netscape".

Chrome, Firefox, IE, Safari, and Opera all return appCodeName "Mozilla".

The Browser Names

```
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
"Name is " + navigator.appName + ". Code name is " + navigator.appCodeName;
</script>
```

What is the name(s) of your browser?

Try it

Name is Netscape
Code name is Mozilla

Browser Version

```
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = navigator.appVersion;
</script>
```

Display version information about the browser.

Try it

5.0 (Windows NT 10.0; WOW64; Trident/7.0; .NET4.0C; .NET4.0E; .NET CLR 2.0.50727; .NET CLR 3.0.30729; .NET CLR 3.5.30729; InfoPath.3; rv:11.0) like Gecko

Browser Platform

```
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = navigator.platform;
</script>
```

Display browser platform.

Try it

Win32

9. JavaScript JSON

JSON: 데이터를 주고받는 공통 표현 형식

JavaScript Object Notation — 가볍고, 텍스트 기반이며, 언어에 독립적인 데이터 교환 형식

① 등장 동기: 서로 다른 환경의 데이터 교환

메모리의 객체를 그대로 네트워크로 보낼 수는 없다.
구조화된 데이터를 공통 텍스트 형식으로 표현 → 전송 · 저장 → 복원



JavaScript 객체 표기에서 유래했지만, 여러 언어에서 사용된다.

XML과 JSON의 표현 예시 (동일한 데이터)

XML

```
<user>
  <name>Kim</name>
  <age>20</age>
</user>
```

JSON

```
{
  "name": "Kim",
  "age": 20
}
```

태그를 반복하지 않아 간결한 표현 가능.

② 표현 방법: 객체 · 배열 · 기본 값

JSON 예제

```
{
  "name": "Kim",
  "age": 20,
  "active": true,
  "courses": ["Web", "AI"],
  "address": { "city": "Seoul" },
  "note": null
}
```

키 (문자열) · 숫자 값 · 불리언 값 · 배열 (순서 있는 목록) · 중첩 객체 (객체) · null 값

JSON의 기본 규칙

1. 객체 { }: 이름-값 쌍
   배열 [ ]: 순서 있는 목록
2. 값: string · number · boolean · null · object · array
3. 키와 문자열은 큰따옴표 (")
   항목 구분은 쉼표 (,)
4. 주석 · 마지막 쉼표 · undefined · 함수 · NaN · Infinity 불가

최상위 값은 객체 · 배열뿐 아니라 기본 값도 가능하다.

③ JSON의 장점

- **간결한 문법** — 읽고 작성하기 쉬움
- **언어 간 호환** — 다양한 파서 · 라이브러리
- **구조 표현** — 객체 · 배열의 중첩 지원
- **웹 API에 적합** — 데이터와 화면 표현 분리

주요 활용 사례: API 요청/응답 · 설정 파일 · 데이터 저장

④ JSON의 단점과 주의점

- **주석 없음** — 설명을 문서로 관리
- **타입 제한** — 날짜 · 바이너리는 별도 표현 약속
- **스키마 내장 없음** — 구조 · 의미 검증은 별도 도구 사용
- **텍스트 비용** — 바이너리 형식보다 크기 · 파싱 비용이 클 수 있음

큰 정수의 정밀도는 수신 언어 · 파서에 주의해야 한다.

⑤ JavaScript 객체 ≠ JSON 텍스트 (직렬화 · 역직렬화 과정)

JavaScript 객체 (메모리의 객체)

```
const obj = {
  name: "Kim",
  age: 20
};
```

JSON.stringify(obj) → 직렬화 (객체 → JSON 텍스트)

JSON 텍스트 (네트워크로 전송하는 문자열)

```
{
  "name": "Kim",
  "age": 20
}
```

JSON.parse(text) → 역직렬화 (JSON 텍스트 → 객체)

JavaScript 객체 (복원된 객체)

```
{
  name: "Kim",
  age: 20
}
```



JSON은 실행 코드가 아니라 데이터 형식이다: 문자열로 전달하고 파서로 읽는다.

공식 참고: RFC 8259 · ECMA-404

9. JavaScript JSON

- **What is JSON ?**
  - JSON stands for JavaScript Object Notation
  - JSON is a lightweight data-interchange format (Server <-> Client)
  - JSON is language independent
  - JSON is an easier-to-use alternative to XML.

XML Example — **XML Document**

```
<employees>
    <employee>
        <firstName>John</firstName> <lastName>Doe</lastName>
    </employee>
    <employee>
        <firstName>Anna</firstName> <lastName>Smith</lastName>
    </employee>
    <employee>
        <firstName>Peter</firstName> <lastName>Jones</lastName>
    </employee>
</employees>
```

JSON Example — **JASON object**

```
{"employees":[
    {"firstName":"John", "lastName":"Doe"},
    {"firstName":"Anna", "lastName":"Smith"},
    {"firstName":"Peter", "lastName":"Jones"}
]}
```

Quiz : https://www.w3schools.com/quiztest/quiztest.asp?qtest=JavaScript

- **JSON vs. XML**
  - **Much like XML Because**
    - ⇔ Both JSON and XML are "self describing" (human readable)
    - ⇔ Both JSON and XML are hierarchical (values within values)
    - ⇔ Both JSON and XML can be parsed and used by lots of programming languages
    - ⇔ Both JSON and XML can be fetched with an XMLHttpRequest
  - **Much Unlike XML Because**
    - ⇔ JSON doesn't use end tag
    - ⇔ JSON is shorter
    - ⇔ JSON is quicker to read and write
    - ⇔ JSON can use arrays
    - ⇔ XML has to be parsed with an XML parser. JSON can be parsed by a standard JavaScript function.
- **Why JSON ?**
  - **For AJAX applications, JSON is faster and easier than XML:**

Using XML

- Fetch an XML document
- Use the XML DOM to loop through the document
- Extract values and store in variables

Using JSON

- Fetch a JSON string
- JSON.Parse the JSON string

JSON - Evaluates to JavaScript Objects

- The JSON format is **syntactically identical** to the code for creating JavaScript objects.
- Because of this similarity, instead of using a parser (like XML does), a JavaScript program can use **standard JavaScript functions** to convert JSON data into native JavaScript objects.

② Receiving the data

```
<!DOCTYPE html>
<html>
<body>

<h2>JSON Object Creation in JavaScript</h2>

<p id="demo"></p>

<script>
var text = '{"name":"John Johnson","street":"Oslo West 16","phone":"555 1234567"}';

var obj = JSON.parse(text);

document.getElementById("demo").innerHTML =
obj.name + "<br>" +
obj.street + "<br>" +
obj.phone;
</script>

</body>
</html>
```

obj

name : John Johnson
street : Oslo West 16
phone : 555 1234567

encoding
decoding

property value

key value

객체

associative array ➔ object

decoding

출력 → 브라우저 화면이 바뀜

JSON Object Creation in JavaScript

John Johnson
Oslo West 16
555 1234567

① Sending the data

```
<?php
$myObj->name = "John";
$myObj->age = 30;
$myObj->city = "New York";

$myJSON = json_encode($myObj);

echo $myJSON;
?>
```

객체

(객체를 associative array로 변환)

associative array

출력 → 브라우저에 전달됨

```
{"name":"John","age":30,"city":"New York"}
```

- **Decoding** : JSON.parse() : Javascript Built-in Object : associative array ➔ Javascript object
- JSON.stringify(): JavaScript에서 Object를 JSON으로 변환
- **Encoding** : json_encode() : PHP library : : object ➔ associative array

– JSON Arrays

⇔ JSON arrays are written inside square brackets.

⇔ Just like JavaScript, a JSON array can contain multiple objects:

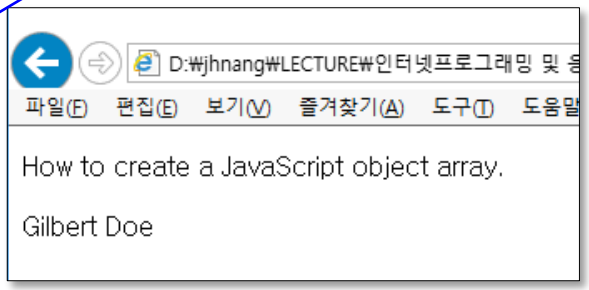
```
<!DOCTYPE html>
<html>
<body>

<p>How to create a JavaScript object array.</p>

<p id="demo"></p>

<script>
var employees = [
    {
    "firstName":"John",
    "lastName":"Doe"
    },
    {
    "firstName":"Anna",
    "lastName":"Smith"
    },
    {
    "firstName":"Peter",
    "lastName":"Jones"
    }
];
employees[0]["firstName"] = "Gilbert";
document.getElementById("demo").innerHTML =
employees[0].firstName + " " + employees[0].lastName;
</script>

</body>
</html>
```



JSON.stringify

```
let student = {
    name:'John',
    age: 30,
    isAdmin: false,
    courses: ['html','css','js'],
    wife:null
}


let json = JSON.stringify(student);


console.log(typeof json); // string
console.log(json);
/* JSON으로 인코딩된 객체:
{
  "name": "John",
  "age": 30,
  "isAdmin": false,
  "courses": ["html", "css", "js"],
  "wife": null
}
*/
```

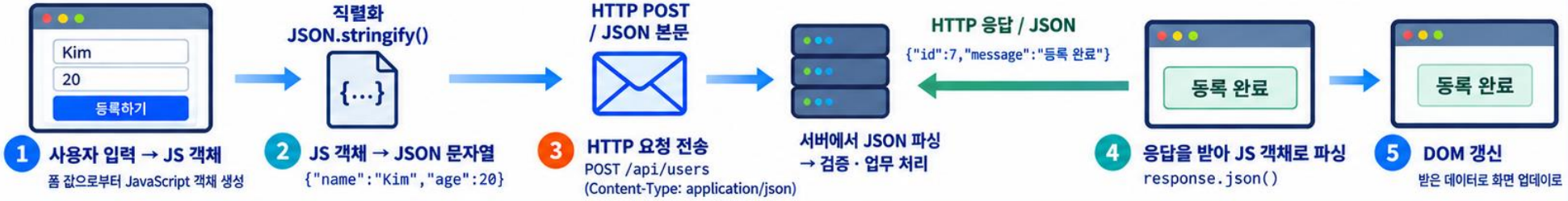
(JavaScript) JSON.stringify() : Object → JSON

JSON을 담은 HTTP Request 보내기

fetch() + POST + JSON.stringify(): 객체를 JSON 본문으로 전송

1 전체 흐름

HTTP는 전송 규칙, JSON은 본문 데이터의 표현 형식입니다.



2 브라우저 JavaScript 예제 (fetch)

같은 출처 /api/users 서버 API와 #msg 요소가 준비된 예제

```
1  async function sendUser() {
2    const user = { name: "Kim", age: 20 };
3    try {
4      const res = await fetch("/api/users", {
5        method: "POST",
6        headers: {
7          "Content-Type": "application/json",
8          "Accept": "application/json"
9        },
10       body: JSON.stringify(user)
11     });
12     if (!res.ok) throw new Error("HTTP " + res.status);
13     const result = await res.json();
14     document.querySelector("#msg").textContent =
15       result.message;
16   } catch (err) {
17     console.error(err);
18 }
   sendUser();
```

await
응답을 기다리는 동안
브라우저 전체를 멈추지 않음

JSON 문자열을
요청 본문으로 전송

3 실제 HTTP 메시지 (HTTP/1.1 개념 예)

요청 (브라우저 → 서버)

```
POST /api/users HTTP/1.1
Host: example.com
Content-Type: application/json
Accept: application/json

{"name":"Kim","age":20}
```

빈 줄 아래가 요청 본문 · 기타 헤더는 생략

응답 (서버 → 브라우저)

```
HTTP/1.1 201 Created
Content-Type: application/json

{"id":7,"message":"등록 완료"}
```

result.message → "등록 완료"

동록 완료

화면의 #msg 요소에 표시

4 코드의 핵심과 주의점

- **method: POST** — 본문 데이터를 보내는 요청
- **Content-Type** — 보내는 본문이 JSON임을 알림
- **JSON.stringify()** — JS 객체 → JSON 문자열
- **await res.json()** — 응답 본문을 읽고 JS 값으로 파싱

- Accept는 원하는 응답 형식이며, 서버가 실제 형식을 결정합니다.
- fetch는 404 · 500에도 이행될 수 있으므로 res.ok 확인이 필요합니다.
- 서버가 JSON 응답을 준다는 전제: 빈 응답 · 다른 형식은 별도 처리해야 합니다.
- 다른 출처로 application/json POST: 보통 CORS preflight(OPTIONS)와 서버 허용이 필요합니다.
- GET 요청은 fetch에서 body를 넣지 않으며, 보통 URL 쿼리 사용합니다.

객체 → JSON 직렬화 → HTTP 전송 → 서버 처리 → JSON 응답 → 화면 갱신

참고: MDN Fetch API

- **JSON Http Request**
  - A common use of JSON is **to read data from a web server, and display the data in a web page.**

```
<div id="id01"></div>

<script>
var xmlhttp = new XMLHttpRequest();
var url = "myTutorials.txt";

xmlhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
        var myArr = JSON.parse(this.responseText);
        myFunction(myArr);
    }
};
xmlhttp.open("GET", url, true);
xmlhttp.send();

function myFunction(arr) {
    var out = "";
    var i;
    for(i = 0; i < arr.length; i++) {
        out += '<a href="' + arr[i].url + '">' +
        arr[i].display + '</a><br>';
    }
    document.getElementById("id01").innerHTML = out;
}
</script>
```

Ajax

object array

출력

Javascript 출력 (HTML 문서)을 화면에 rendering한 결과

HTML Tutorial
CSS Tutorial
JavaScript Tutorial
jQuery Tutorial
SQL Tutorial
PHP Tutorial
XML Tutorial

myTutorials.txt

```
[ { "display": "HTML Tutorial",
    "url":"http://www.w3schools.com/html/default.asp" },
  { "display": "CSS Tutorial",
    "url": "http://www.w3schools.com/css/default.asp" },
  { "display": "JavaScript Tutorial",
    "url": "http://www.w3schools.com/js/default.asp" },
  { "display": "jQuery Tutorial",
    "url": "http://www.w3schools.com/jquery/default.asp" },
  { "display": "SQL Tutorial",
    "url": "http://www.w3schools.com/sql/default.asp" },
  { "display": "PHP Tutorial",
    "url": "http://www.w3schools.com/php/default.asp" },
  { "display": "XML Tutorial",
    "url": "http://www.w3schools.com/xml/default.asp" }
]
```

(이런 JSON 형식으로 서버에서 PHP 프로그램이 생성하여 Client에게 보냄)

- **JSON SQL Example**
  - **Client Javascript file**

Customers.html

```
<!DOCTYPE html>
<html>
<body>

<h1>Customers</h1>
<div id="id01"></div>

<script>
var xmlhttp = new XMLHttpRequest();
var url = "http://www.w3schools.com/js/customers_mysql.php";

xmlhttp.onreadystatechange=function() {
    if (this.readyState == 4 && this.status == 200) {
        myFunction(this.responseText);
    }
}
xmlhttp.open("GET", url, true);
xmlhttp.send();
```

```
function myFunction(response) {
    var arr = JSON.parse(response);
    var i;
    var out = "<table>";

    for(i = 0; i < arr.length; i++) {
        out += "<tr><td>" +
        arr[i].Name +
        "</td><td>" +
        arr[i].City +
        "</td><td>" +
        arr[i].Country +
        "</td></tr>";
    }
    out += "</table>";
    document.getElementById("id01").innerHTML = out;
}
</script>

</body>
</html>
```


[{ "Name" : "Alfreds Futterkiste", "City" : "Berlin", "Country" : "Germany" }, { "Name" : "Berglunds snabbköp", "City" : "Luleå", "Country" : "Sweden" }, { "Name" : "Centro comercial Moctezuma", "City" : "México D.F.", "Country" : "Mexico" }, { "Name" : "Ernst Handel", "City" : "Graz", "Country" : "Austria" }, { "Name" : "FISSA Fabrica Inter. Salchichas S.A.", "City" : "Madrid", "Country" : "Spain" }, { "Name" : "Galería del gastrónomo", "City" : "Barcelona", "Country" : "Spain" }, { "Name" : "Island Trading", "City" : "Cowes", "Country" : "UK" }, { "Name" : "Königlich Essen", "City" : "Brandenburg", "Country" : "Germany" }, { "Name" : "Laughing Bacchus Wine Cellars", "City" : "Vancouver", "Country" : "Canada" }, { "Name" : "Magazzini Alimentari Riuniti", "City" : "Bergamo", "Country" : "Italy" }, { "Name" : "North/South", "City" : "London", "Country" : "UK" }, { "Name" : "Paris spécialités", "City" : "Paris", "Country" : "France" }, { "Name" : "Rattlesnake Canyon Grocery", "City" : "Albuquerque", "Country" : "USA" }, { "Name" : "Simons bistro", "City" : "København", "Country" : "Denmark" }, { "Name" : "The Big Cheese", "City" : "Portland", "Country" : "USA" }, { "Name" : "Vaffeljernet", "City" : "Århus", "Country" : "Denmark" }, { "Name" : "Wolski Zajazd", "City" : "Warszawa", "Country" : "Poland" }]

Customers

Alfreds Futterkiste	Berlin	Germany
Berglunds snabbköp	Luleå	Sweden
Centro comercial Moctezuma	México D.F.	Mexico
Ernst Handel	Graz	Austria
FISSA Fabrica Inter. Salchichas S.A.	Madrid	Spain
Galería del gastrónomo	Barcelona	Spain
Island Trading	Cowes	UK
Königlich Essen	Brandenburg	Germany
Laughing Bacchus Wine Cellars	Vancouver	Canada
Magazzini Alimentari Riuniti	Bergamo	Italy
North/South	London	UK
Paris spécialités	Paris	France
Rattlesnake Canyon Grocery	Albuquerque	USA
Simons bistro	København	Denmark
The Big Cheese	Portland	USA
Vaffeljernet	Århus	Denmark
Wolski Zajazd	Warszawa	Poland

– Server PHP file

www.w3schools.com/js/customers_mysql.php

```php
<?php
header("Access-Control-Allow-Origin: *");
header("Content-Type: application/json; charset=UTF-8");

$conn = new mysqli("myServer", "myUser", "myPassword", "Northwind");

$result = $conn->query("SELECT CompanyName, City, Country FROM Customers");

$outp = "[";
while($rs = $result->fetch_array(MYSQLI_ASSOC)) {
    if ($outp != "[") {$outp .= ",";}
    $outp .= '{"Name":"'  . $rs["CompanyName"] . '",';
    $outp .= '"City":"'   . $rs["City"]        . '",';
    $outp .= '"Country":"'. $rs["Country"]     . '"}';
}
$outp .="]";

$conn->close();

echo($outp);
?>
```

string concatenation

DB 검색 결과를 JSON 형식으로 인코딩

```
[ { "Name" : "Alfreds Futterkiste", "City" : "Berlin", "Country" : "Germany" }, { "Name" : "Berglunds snabbköp", "City" : "Luleå", "Country" : "Sweden" }, { "Name" : "Centro comercial Moctezuma", "City" : "México D.F.", "Country" : "Mexico" }, { "Name" : "Ernst Handel", "City" : "Graz", "Country" : "Austria" }, { "Name" : "FISSA Fabrica Inter. Salchichas S.A.", "City" : "Madrid", "Country" : "Spain" }, { "Name" : "Galería del gastrónomo", "City" : "Barcelona", "Country" : "Spain" }, { "Name" : "Island Trading", "City" : "Cowes", "Country" : "UK" }, { "Name" : "Königlich Essen", "City" : "Brandenburg", "Country" : "Germany" }, { "Name" : "Laughing Bacchus Wine Cellars", "City" : "Vancouver", "Country" : "Canada" }, { "Name" : "Magazzini Alimentari Riuniti", "City" : "Bergamo", "Country" : "Italy" }, { "Name" : "North/South", "City" : "London", "Country" : "UK" }, { "Name" : "Paris spécialités", "City" : "Paris", "Country" : "France" }, { "Name" : "Rattlesnake Canyon Grocery", "City" : "Albuquerque", "Country" : "USA" }, { "Name" : "Simons bistro", "City" : "København", "Country" : "Denmark" }, { "Name" : "The Big Cheese", "City" : "Portland", "Country" : "USA" }, { "Name" : "Vaffeljernet", "City" : "Århus", "Country" : "Denmark" }, { "Name" : "Wolski Zajazd", "City" : "Warszawa", "Country" : "Poland" }
]
```

Client에게 전달되는 string, printed by “echo()”

– 참고 : A Styled Version

Customers.html

```
<!DOCTYPE html>
<html>

<head>
<style>
h1 {
    border-bottom: 3px solid #cc9900;
    color: #996600;
    font-size: 30px;
}
table, th , td {
    border: 1px solid grey;
    border-collapse: collapse;
    padding: 5px;
}
table tr:nth-child(odd) {
    background-color: #f1f1f1;
}
table tr:nth-child(even) {
    background-color: #ffffff;
}
</style>
</head>

<body>

<h1>Customers</h1>
<div id="id01"></div>

<script>
var xmlhttp = new XMLHttpRequest();
var url = "http://www.w3schools.com/js/customers_mysql.php";
```

Customers

Alfreds Futterkiste	Berlin	Germany
Berglunds snabbköp	Luleå	Sweden
Centro comercial Moctezuma	México D.F.	Mexico
Ernst Handel	Graz	Austria
FISSA Fabrica Inter. Salchichas S.A.	Madrid	Spain
Galería del gastrónomo	Barcelona	Spain
Island Trading	Cowes	UK
Königlich Essen	Brandenburg	Germany
Laughing Bacchus Wine Cellars	Vancouver	Canada
Magazzini Alimentari Riuniti	Bergamo	Italy
North/South	London	UK
Paris spécialités	Paris	France
Rattlesnake Canyon Grocery	Albuquerque	USA
Simons bistro	København	Denmark
The Big Cheese	Portland	USA
Vaffeljernet	Århus	Denmark
Wolski Zajazd	Warszawa	Poland

- **PHP JASON :**
  - **Encode : json_encode()**

Associative array

```
<!DOCTYPE html>
<html>
<body>

<?php
$age = array("Peter"=>35, "Ben"=>37, "Joe"=>43);

echo json_encode($age);
?>

</body>
</html>
```

```
{"Peter":35,"Ben":37,"Joe":43}
```

Indexed array

```
<!DOCTYPE html>
<html>
<body>

<?php
$cars = array("Volvo", "BMW", "Toyota");

echo json_encode($cars);
?>

</body>
</html>
```

```
["Volvo","BMW","Toyota"]
```

– Decode : json_decode()

```
<!DOCTYPE html>
<html>
<body>

<?php
$jsonobj = '{"Peter":35,"Ben":37,"Joe":43}';

var_dump(json_decode($jsonobj));
?>

</body>
</html>
```

```
object(stdClass)#1 (3) { ["Peter"]=> int(35) ["Ben"]=> int(37) ["Joe"]=> int(43) }
```

http://cspro.sogang.ac.kr/~jhnang/cgi-bin/php_json_test.php

```
<!DOCTYPE html>
<html>
<body>

<?php
$jsonobj = '{"Peter":35,"Ben":37,"Joe":43}';

$arr = json_decode($jsonobj, true);

echo $arr["Peter"];
echo $arr["Ben"];
echo $arr["Joe"];
?>

</body>
</html>
```

- access the values from the PHP associative array

```
<!DOCTYPE html>
<html>
<body>

<?php
$jsonobj = '{"Peter":35,"Ben":37,"Joe":43}';

$obj = json_decode($jsonobj);

echo $obj->Peter;
echo $obj->Ben;
echo $obj->Joe;
?>

</body>
</html>
```

- access the values from the PHP object:

```
353743
```

```php
<?php

header('Content-type: text/html');
echo json_encode(
    array(
        array('name' => 'Kim', 'grade' => '4.4'),
        array('name' => 'Park', 'grade' => '3.7'),
        array('name' => 'Lee', 'grade' => '4.0')
    )
);
```

```
[{"name":"Kim","grade":"4.4"},
{"name":"Park","grade":"3.7"},
{"name":"Lee","grade":"4.0"}]
```

```php
<?php

//json encode 변환
$json = json_encode(array(
        array('name' => 'Kim', 'grade' => '4.4'),
        array('name' => 'Park', 'grade' => '3.7'),
        array('name' => 'Lee', 'grade' => '4.0')
    )
);

//출력1
echo $json;

//json decode 변환
$student = json_decode($json);

//출력2
echo "<br/>";
echo "<br/>";
echo $student[1]->name." : ".$student[1]->grade;
echo "<br/>";
echo $student[2]->name." : ".$student[2]->grade;
```

```
[{"name":"Kim","grade":"4.4"},
{"name":"Park","grade":"3.7"},
{"name":"Lee","grade":"4.0"}]

Park : 3.7
Lee : 4.0
```

JSON.stringify()와 json_encode() 비교

정확한 이름: JavaScript는 JSON.stringify() / PHP는 json_encode()

JSON.stringfy()는 오타. JSON.encode()는 표준 JavaScript 함수가 아님:
특정 라이브러리에는 있을 수 있음. (이 강의에서는 PHP의 json_encode()를 뜻한 경우의 비교입니다.)

JavaScript: JSON.stringify()

JS 값 → JSON 문자열

```
const obj = { name: "Kim", age: 20 };
const text = JSON.stringify(obj);
// "{"name":"Kim","age":20}"
```

브라우저 · Node.js 등에서 사용

역직렬화(복원): `JSON.parse(text)`

공통 목적: 직렬화

동일한 JSON 형식

JSON.stringify() → `{"name":"Kim", "age":20}` ← json_encode()

PHP: json_encode()

PHP 값 → JSON 문자열

```
$data = ["name" => "Kim", "age" => 20];
$text = json_encode($data);
// {""name":"Kim","age":20}"
```

웹 서버 등 PHP 실행 환경에서 사용

역직렬화(복원): `json_decode($text, true)`
이 예제는 연관 배열로 복원

항목	JavaScript: JSON.stringify()	PHP: json_encode()
입력	JavaScript 객체 · 배열 등	PHP 배열 · 객체 등
출력	JSON 문자열 (값에 따라 undefined 가능)	JSON 문자열 / 실패 시 false
옵션	replacer · space	flags · depth
실패	순환 참조 · BigInt 동은 예외 (TypeError 등)	JSON_THROW_ON_ERROR 사용 시 예외

PHP: 연속 숫자 키 배열은 JSON 배열,
연관 배열은 JSON 객체로 표현

3 실제 통신 예제 (브라우저 ↔ PHP 서버)

브라우저 (JavaScript) → JS 객체 `{ name: "Kim", age: 20 }` → JSON.stringify() → HTTP 요청 (애플리케이션/json) `{"name":"Kim","age":20}` → PHP 서버 → json_encode() → HTTP 응답 (애플리케이션/json) `{"name":"Kim","age":20}` → JSON.parse() 또는 response.json() → 브라우저 (JavaScript)

두 함수 자체가 HTTP 전송을 하지는 않음: 전송은 fetch · 웹 서버 등이 담당합니다.

이름은 다르지만 역할은 비슷하다: 각 언어의 값을 공통 JSON 텍스트로 바꾼다.

참고: MDN JSON.stringify() | PHP Manual: json_encode()